Int. J. Fuzzy Syst.
https://doi.org/10.1007/s40815-025-02197-5

CoEDMTVideo: Collaborative Edge Computing and Optimal Ensemble Deep Q-Learning Network for Distributed Model Training in IoT Video Stream Environments

Shaharyar Alam Ansari[1] · **Mohammad Luqman**[2] · **Mohd Anas Wajid**[3] · **Mohammad Saif Wajid**[4] · **Shahnawaz Ahmad**[1]

Received: 1 May 2024 / Revised: 25 October 2025 / Accepted: 28 October 2025
© The Author(s) under exclusive licence to Taiwan Fuzzy Systems Association 2026

Abstract The integration of Internet of Things (IoT) technology with video applications enables real-time monitoring and decision-making across various contexts. These applications utilize interconnected devices equipped with cameras, sensors, and communication capabilities to collect visual information. Despite their revolutionary impact on multiple industries, IoT video applications face significant challenges, such as high latency, energy consumption, and the complexity of managing large volumes of video data. Traditional Deep Learning (DL) models contribute to high communication latency, creating a need for distributed solutions. To address these issues, the Collaborative Edge Distributed Model Training Video (CoEDMTVideo) framework is introduced. This framework integrates collaborative edge computing and Deep Reinforcement Learning (DRL) for distributed model training in IoT video streaming environments. The Video Streaming System (VSS) within CoEDMTVideo utilizes the Optimal Ensemble Deep Q-learning Network (OEDQNet) for distributed model training in the IoT context. Initially, a multi-driven Collaborative Edge Computing (CEC) platform is deployed to enhance computing capacity and reduce network communication overhead. We further introduce a light-weight fuzzy decision layer that biases action selection toward QoS-aware choices, improving stability under dynamic bandwidth and latency conditions. This platform includes numerous IoT terminals (ITs), Collaborative Edge nodes (CENs), and a Central Server (CS). The OEDQNet algorithm improves distributed model training by leveraging an ensemble learning technique for robust and reliable decision-making and by combining ensemble learning with deep Q-learning to optimize system performance in dynamic IoT environments. Simulations using the YouTube dataset demonstrate that the proposed framework enhances video processing efficiency and reduces energy consumption. The simulation results show that the proposed CoEDMTVideo achieves significant improvements, demonstrating its overall effectiveness in IoT video applications.

Keywords IoT · Collaborative edge computing · Reinforcement learning · Video processing

✉ Mohammad Luqman
luqman.geeky@gmail.com

[1] School of Computer Science Engineering and Technology, Bennett University, Greater Noida, India

[2] Department of Computer Science, Aligarh Muslim University, Aligarh, India

[3] Institute for the Future of Education, Tecnológico de Monterrey, Monterrey, Mexico

[4] Southern Methodist University, Dallas, TX, USA

1 Introduction

The integration of Internet of Things (IoT) technology with video applications has opened up new horizons for various industries, offering innovative solutions for surveillance, monitoring, and data analysis [1]. The concept of IoT in video entails the integration of interconnected devices furnished with cameras and sensors to capture, transmit, and analyze visual data instantaneously. This innovation facilitates the development of intelligent environments where video data play a vital role in refining decision-making processes, streamlining operations, and enhancing overall efficiency. In the domain of IoT video applications, Edge computing (EC), Mobile Edge Computing (MEC), and Collaborative Edge Computing (CEC) have emerged as fundamental technologies. EC entails processing data in proximity to its source, thereby minimizing latency and conserving bandwidth by executing computations at the network's edge. MEC extends

Published online: 07 April 2026

this concept to mobile devices, enabling real-time processing and decision-making capabilities directly on smartphones and tablets. Collaborative Edge Computing (CEC) takes this a step further by fostering collaboration among edge devices, enhancing computational power and enabling more sophisticated video processing tasks [2]. These technologies offer significant advantages in IoT environments, allowing for faster response times, and reduced network congestion by minimizing the need to transmit sensitive video data over long distances. By leveraging the power of edge computing, IoT video applications can achieve greater scalability, reliability, and efficiency for more connected and intelligent future [3,4]. Features of EC, MEC, and CEC features are shown in Table 1.

Over the past years, CEC emerges as a compelling solution for IoT video applications due to its ability to address the challenges associated with processing and analyzing vast amounts of video data in real time. Conventional edge computing paradigms often encounter limitations with individual edge devices possessing restricted computational capabilities, thus presenting challenges in efficiently executing intricate video processing tasks [6,7]. However, CEC leverages the collective computing power of interconnected edge nodes to collaboratively process and analyze video data. By distributing processing tasks across multiple edge devices, CEC enables parallel execution of algorithms, reducing latency and improving overall system performance [8]. Moreover, CEC fosters collaboration among edge nodes, allowing them to share resources and insights, which enhances the scalability and adaptability of IoT video applications [9]. Despite the promising potential of CEC for IoT video applications, several research gaps persist in existing algorithms and methodologies. A major hurdle involves crafting effective resource allocation and task scheduling strategies to optimize edge computing resource utilization, while concurrently minimizing latency and energy consumption. Existing algorithms may falter in scalability and adaptability, particularly in dynamic IoT environments where network conditions and workload distributions fluctuate unpredictably. Rectifying these research gaps is imperative to fully harnessing CECs potential for IoT video applications and materializing the vision of a cohesive, intelligent edge computing ecosystem.

Existing studies in CEC utilizing DL algorithms and RL techniques have shown promising results in addressing the challenges of IoT applications [10,11]. These approaches leverage the computational power of edge devices to perform complex tasks in real time. DL algorithms, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), enable edge devices to extract meaningful features from raw data, facilitating accurate and efficient analysis [12,13]. RL algorithms offer a framework enabling edge devices to acquire optimal decision-making policies

Table 1 Features of computing paradigms [5]

Features	Edge computing (EC)	Mobile edge computing (MEC)	Collaborative edge computing (CEC)
Location	Near data sources	Proximity to mobile users and devices	Geographically distributed, interconnected
Infrastructure	Single edge device or gateway	Network of edge servers and devices	Network of interconnected edge nodes
Resource pooling	Limited resources	Enhanced resource pooling and sharing	Extensive resource pooling and sharing
Latency reduction	Local processing	Mobile-centric latency reduction	Latency reduction through collaboration
Use cases	Real-time IoT video analytics	Mobile video streaming, AR/VR applications	Multi-user video conferencing, video content delivery
Communication	Direct communication with data source	Proximity-based communication	Inter-node communication for collaborative processing
Scalability	Limited scalability due to single device	Scalable due to distributed infrastructure	Highly scalable with interconnected nodes
Deployment flexibility	Single point deployment	Flexible deployment near mobile users	Flexible deployment across interconnected nodes
Video processing capability	Localized processing	Processing near mobile users	Distributed processing with collaborative algorithms
Network connectivity	Wired or wireless networks	Cellular, Wi-Fi, and other wireless networks	Interconnected edge nodes with various network connections
Mobility support	Static environments	Supports mobile users and devices	Supports mobility and dynamic network topologies
Heterogeneity	Limited device heterogeneity	Supports heterogeneous mobile devices	Embraces heterogeneity across interconnected nodes
Data handling	Real-time video data processing	On-device video streaming, real-time analytics	Collaborative video processing and analytics
Network management	Simple edge management	Centralized and distributed network management	Dynamic network management across interconnected nodes

via trial and error, adjusting to evolving requirements and dynamic environments. However, these methods encounter limitations in IoT applications. The resource-constrained nature of edge devices restricts the complexity and scale of deployable DL models, resulting in performance degradation and diminished accuracy [14]. Additionally, the decentralized nature of edge computing introduces challenges in model coordination, synchronization, and training, hindering the scalability and efficiency of collaborative learning approaches [15,16]. Addressing these limitations is crucial for fully harnessing the capabilities of collaborative edge computing in IoT applications and materializing the vision of a unified, intelligent edge ecosystem.

Motivated by the above facts, this work addresses the problems of video processing tasks between the collaborative edge nodes and central server in the IoT environmental conditions. We presented a Video Streaming System (VSS) using Optimal Ensemble Deep Q-Learning Network (OEDQLNet) for distributed model training. OEDQLNet aims to improve the decision-making process in video processing tasks by leveraging deep Q-learning techniques. By training neural networks to approximate the Q values associated with different actions in a video processing environment, OEDQLNet enables intelligent decision-making based on learned policies and reward signals. The main contributions of our manuscript are structured as follows:

- We present a Collaborative Edge Distributed Model Training Video (CoEDMTVideo) framework that integrates CEC and DRL for distributed model training in IoT video streaming environments.
- CoEDMTVideo framework utilizes a multi-driven CEC structure to enhance video processing tasks with efficient computing capacity and reduced data communication overhead and in IoT environments.
- We present a Video Streaming System (VSS) that employs the Optimal Ensemble Deep Q-learning Network (OEDQNet) for distributed model training in the CoEDMTVideo framework, aiming to improve decision-making processes in video processing tasks through deep Q-learning techniques.
- OEDQNet model uses optimal solutions to optimize the allocation of computational resources, storage capacity, and bandwidth in IoT video streaming scenarios, aiming to minimize latency and energy consumption while ensuring efficient video streaming.
- Simulation outcomes highlight the framework's ability that obtains minimized latency, reduced energy consumption, and efficient streamline video processing tasks, ultimately leading to significantly improve decision-making processes in dynamic IoT settings.
- A **fuzzy logic-assisted action selection** layer that fuses QoS-aware linguistic rules with ensemble Deep Q-learning, yielding robust decisions under variable network conditions.

2 Related Works

This section provides a brief overview of recent existing studies, typically summarizes key findings, methodologies, advantages, and disadvantages. The previous works are relevant to the domain of distributed computing resources, artificial intelligence (AI), edge computing, and IoT discussed as follows:

Li et al. [17] primarily delve into the integration of DL techniques with EC (DL-EC) paradigms to enhance IoT applications. Utilizing DL at the edge offers the advantage of real-time processing and analysis of IoT data, facilitating prompt insights and actions without necessitating the transmission of extensive raw data to centralized cloud servers. By deploying DL models closer to IoT devices through edge computing resources, latency is minimized, bandwidth is preserved, and privacy is enhanced through local processing of sensitive data. Nonetheless, implementing DL for IoT with edge computing poses challenges, including the constrained computational and storage capacities of edge devices, which may limit the complexity and scalability of DL models deployed at the edge. Sun et al. [18] focused on developed innovative solutions that leverage edge computing capabilities using deep neural network (DNN) to enhance mobile video processing tasks. The advantage of employing edge computing in mobile video processing lies in its ability for offload computational tasks to nearby edge servers, thereby reducing latency, conserving battery life, and improving overall system performance. By leveraging the proximity of edge servers to mobile devices, mVideo systems perform real-time video analysis, encoding, transcoding, and streaming video conferencing, augmented reality, and mobile surveillance. However, certain challenges exist in implementing mVideo systems. These challenges include the need to optimize resource allocation and scheduling algorithms to efficiently utilize edge computing resources and meet the stringent latency and quality of service requirements of mobile video applications.

Wu et al. [19] focused on integration of DRL-based Markov Decision Process (DRL-MDP) and techniques with MEC architectures to optimize video quality selection and radio bearer control in short video applications. Employing DRL in this context offers the advantage of dynamically learning and optimizing video quality and radio bearer configurations based on fluctuating network conditions, user preferences, and application demands. Through the utilization of MEC resources, DRL-based algorithms can autonomously make real-time decisions to efficiently allocate radio bearers and select appropriate video quality levels,

thereby improving user experience, reducing latency, and conserving bandwidth. Furthermore, DRL-based approaches enable personalized adaptation of video quality and control of radio bearers, accommodating the diverse requirements and preferences of individual users in multi-user settings. However, various challenges must be addressed in implementing DRL-based mechanisms for video quality selection and radio bearer control in MEC-supported short video applications. These challenges include the complexity of training DRL models with large-scale video datasets and real-world network traces, which may require significant computational resources and time. Naveen et al. [20] developed efficient DL Inference Model using CNN (DLIM-CNN) tailored for distributed IoT edge clusters to minimize latency and enhance intelligence at the network edge. Utilizing a low-latency DL inference model in this context offers the advantage of conducting real-time analysis of sensor data and making prompt decisions at the network edge, eliminating the necessity for data transmission to centralized cloud servers. Through the utilization of distributed intelligent IoT edge clusters, low-latency inference models process data locally, thereby reducing latency, conserving bandwidth, and enhancing privacy by minimizing data exposure to external networks. Nonetheless, several challenges exist in implementing low-latency DL inference models for distributed intelligent IoT edge clusters, including the optimization of model architecture and parameters to strike a balance between inference accuracy and computational efficiency, given the limited computational and energy resources available at the network edge.

Tian et al. [21] explore innovative strategies for optimizing micro-service caching in distributed IoT environments leveraging edge computing paradigms through MEC. The use of DRL (DRL) offers several advantages in this context. Firstly, DRL algorithms can adaptively learn and optimize caching policies based on dynamic IoT workload patterns and edge computing resource constraints, thereby improving cache hit rates and reducing latency for IoT applications. Additionally, the distributed cooperative nature of MEC-enabled Distributed Cooperative Micro Service Caching (MEC-DIMA) scheme facilitates efficient resource utilization across edge nodes, enabling scalability and fault tolerance in large-scale IoT deployments. However, the implementation of DRL-based caching mechanisms entails certain challenges. Training DRL models requires significant computational resources and data, which may not always be readily available in resource-constrained edge environments. Moreover, the deployment of complex caching policies may introduce overhead and latency, particularly in scenarios with stringent real-time requirements.

Han and Lee [22] explored the integration of DRL techniques with EC to optimize video processing tasks. Among the methods employed, Deep Q-Network (DQN) algorithms have been utilized to make decisions regarding video processing tasks at the edge. Nevertheless, approaches based DQN face hurdles, such as elevated computational complexity, prolonged training durations, convergence complications, and unstable training dynamics, especially in dynamic and resource-limited edge environments. Additionally, Proximal Policy Optimization (PPO) methods have been considered for their stability and sample efficiency, yet they often necessitate extensive hyper-parameter tuning and may exhibit suboptimal policy convergence in complex edge computing scenarios. Moreover, Actor-Critic models, while offering potential improvements in sample efficiency and convergence, may suffer from policy instability and sensitivity to hyper parameters, limiting their effectiveness in edge computing environments with constrained resources and variable network conditions. Huda and Moh [23] focused extensively on exploring RL techniques for task offloading in diverse computing environments. Researchers have delved into methods leveraging DRL to optimize computation offloading decisions (DRLCO) in various edge computing scenarios, including unnamed aerial vehicles (UAV) swarm-enabled systems tailored for surveillance tasks. Despite the promise of DRL-based approaches, several limitations persist. One notable disadvantage is the computational complexity associated with training DRL models, particularly when considering the real-time constraints inherent in surveillance applications. Additionally, the deployment of such sophisticated algorithms on resource-constrained UAV platforms poses challenges in terms of energy consumption and hardware compatibility, further complicating their practical implementation in edge computing environments.

Qian et al. [24] methods for streaming video analytics include leveraging edge and cloud computing for high accuracy and low latency, employing Deep Neural Networks (DNNs) and heuristic algorithms for pattern recognition and optimization, and utilizing Reinforcement Learning (RL) for adaptive adjustments to dynamic video streaming behaviors. The OSMOTICGATE2 system enhances scalability and adaptability across distributed environments, ensuring real-time analysis and high accuracy. However, limitations include the computational intensity and resource demands of implementing advanced RL systems like MAPPO, challenges in training and generalizing policies across diverse environments, and complexities in deployment and interpretability that may impact practical application in real-world scenarios. Pham-Quoc [25] proposed an approach leverages High-Level Synthesis (HLS) techniques to accelerate H.264 video encoding on FPGA platforms for edge computing applications, aiming to reduce computational burdens inherent in processing such videos on resource-constrained edge devices. HLS converts high-level programming languages like C/C ++ into hardware description languages (Verilog/VHDL), facilitating FPGA programming with opti-

mization techniques such as loop unrolling, pipelining, and array partitioning. This enables efficient utilization of FPGA's parallel processing capabilities to offload and accelerate computationally intensive tasks. However, challenges include the complexity of FPGA development, scalability across different platforms, hardware dependencies, and limitations in generalizing optimizations beyond H.264 encoding, necessitating ongoing research for broader applicability and deployment feasibility in diverse edge computing environments.

Yang et al. [26] introduce the low-latency edge scheduling strategy integrates the Hard Attention Mechanism and Advantage Actor-Critic (A2C) algorithm to enhance task scheduling efficiency. The Hard Attention Mechanism focuses computational resources on critical tasks during resource allocation, optimizing job completion times and reducing task latency. Meanwhile, the A2C algorithm employs deep reinforcement learning to dynamically adjust task scheduling based on real-time conditions, thereby improving overall system responsiveness and efficiency. Despite these advantages, implementing and deploying such advanced models on edge servers pose challenges. It demands significant computational resources and expertise in reinforcement learning, impacting scalability and real-time responsiveness. Additionally, optimizing the model parameters for diverse objectives and ensuring its adaptation to varying edge computing environments remain critical hurdles. Interpretability and scalability concern also arise, as deep reinforcement learning models like HA-A2C may be less transparent compared to traditional heuristic methods, potentially affecting trust and deployment feasibility in practical edge computing applications.

Existing studies highlights the challenges and problems associated with integrating DL techniques with edge computing for IoT applications. These studies emphasize the limitations posed by the computational and storage constraints of edge devices, hindering the scalability and complexity of DL models deployed at the edge as shown in Table 2. Additionally, the significant computational resources and time required for training DRL models in real-time applications, along with the need for extensive hyper-parameter tuning and optimization for video processing tasks, present hurdles in achieving efficient and stable convergence in edge computing environments. Deployment issues, such as energy consumption, and policy instability in dynamic scenarios, further complicate the practical implementation of DL algorithms on edge computing settings.

3 CoEDMTVideo Framework Design

The CoEDMTVideo framework is designed to enhance video processing tasks in IoT environments through the integration of CEC and RL techniques. The primary goal of the CoEDMTVideo framework is to optimize the allocation of computational resources, storage capacity, and network bandwidth across distributed edge devices to improve model training efficiency and scalability while minimizing latency and energy consumption. The framework leverages the Optimal Ensemble Deep Q Learning Network (OEDQLNet) for distributed model training, aiming to enhance decision-making processes through deep Q-learning techniques. By dynamically adjusting resource allocation and facilitating collaborative model training and inference processes among edge devices and a central server, the CoEDMTVideo Framework minimizes latency, optimizes energy consumption, and enables real-time processing of video streaming data.

3.1 Multi-Driven CEC Structure of CoEDMTVideo Framework

Video Streaming System (VSS)-based multi-driven CEC structure holds massive IoT terminals with a broad scope of endless video streams. The term "multi-driven" in IoT video data underscores the complex interplay of various factors that shape the collection, processing, and utilization of video data within IoT environments. Simultaneously, VSS systems face challenges of increasing low-latency response and high energy consumption due to high data volume, communication overhead and complexity. To solve these issues, we integrated IoT, CEC, and RL technologies for video processing tasks. Initially, we employed a multi-driven CEC platform to provide efficient computing capacity and reduce network communication overhead for the CoEDMTVideo framework. Also, it contains a massive number of IoT terminals (ITs), Collaborative Edge nodes (EN), and Central Server (CS), as shown in Fig. 1. Each edge device has limited computational resources and storage capacity. The CS acts as a coordinator responsible for video data management and visualization. The ED and CS collaborate to optimize latency and energy consumption in video streaming.

The key components of the CoEDMTVideo framework are presented as follows:

3.1.1 IoT Terminals

IoT terminals ($ITs = [IT_1, IT_2, ...IT_M,]$), where M is the number of IoT Terminals *(IT),* particularly those tailored for video streaming data which represent a specialized category of devices within the IoT ecosystem. These terminals are equipped with high-definition cameras and communication modules optimized for capturing, processing, and transmitting video data in real-time. Equipped with high-definition cameras featuring physical resolutions of 1080P or 4K and leveraging advanced video coding standards such as H.264 or H.265, these terminals deliver superior image

Table 2 Summary of reviewed methods

Schemes	Data selection	Learning context	General purpose	Application
Li et al. [17]	IoT data processing	DL at the edge	Enhancing IoT applications	IoT
Sun et al. [18]	Mobile video processing	edge computing with DNN	offloading tasks from mobile devices	video conferencing, AR, mobile surveillance
Wu et al. [19]	Dynamic network conditions, user preferences	DRL-based MEC optimization	video quality selection, radio bearer control	Short video applications
Naveen et al. [20]	Sensor data analysis	low-latency DL inference	real-time analysis, decision-making	distributed IoT edge clusters
Tian et al. [21]	IoT workload patterns, edge computing resources	DRL-based micro-service caching optimization	Cache hit rates, latency reduction	Distributed IoT environments
Han and Lee [22]	Video processing tasks	DRL techniques with edge computing	Optimizing video processing	Video processing tasks
Huda and Moh [23]	Computation offloading decisions, UAV swarm-enabled	RL techniques for task	task offloading, surveillance applications	UAV swarm-enabled surveillance systems
Qian et al. [24]	Mobile video processing	multi-agent DRL	video streaming behaviors	surveillance system
Pham-Quoc [25]	Video processing	edge computing with an encoder	target IoT devices with limited energy capacity	IoT
Yang et al. [26]	Workload server data	latency optimization	low-latency task scheduling	IoT and data-driven applications

quality and compression efficiency, ensuring optimal utilization of network bandwidth and storage resources. In addition to video capture and processing capabilities, IoT terminals for video streaming data are equipped with robust communication modules that support wireless communication protocols, such as Ethernet, Wi-Fi, cellular networks, and low-power wide-area networks (LPWANs). These communication channels facilitate seamless transmission of video streams to collaborative edge computing nodes for further analysis.

3.1.2 ENs and CS

Each EN within the CoEDMTVideo framework is equipped with capabilities to acquire IoT video streaming data from local sources, such as monitoring devices deployed within its operational domains, such as city, streets, districts, and countries. ENs engage in collaborative model training and inference, utilizing local computational resources for video analysis tasks. The CS orchestrates this process by aggregating local gradients, weights, and optimization parameters, producing a globally synchronized model representation reflecting insights from distributed video data sources. The data management and optimization layer orchestrate resource allocation, utilization, and optimization across distributed edge devices, ensuring efficient communication and coordination within the CoEDMTVideo framework. It dynamically adjusts resource allocation based on workload demands, system performance metrics, and operational priorities to optimize model training efficiency and scalability, facilitating cohesive distributed model training tasks between the EN and the CS.

3.1.3 Distributed Model Training in Video Processing Tasks

Distributed model training for video processing tasks represents a pivotal component toward harnessing the distributed computational capabilities for management and visualization inherent in IoT video streaming data. This component coordinates the collaborative efforts of EDs and a CS to optimize the model training processes of CoEDMTVideo framework. In this work, we focused mainly on the deep learning network model of video streaming tasks in an IoT environment.

3.2 CEC-Based Distributed Model Training of CoEDMTVideo Framework

In IoT applications, most of the DL models used distributed solutions on edge servers, which leads to high network communication overhead, low-latency response, and high energy consumption. Aiming at these facts, we proposed a VSS system with an Optimal Ensemble Deep Q-learning Network

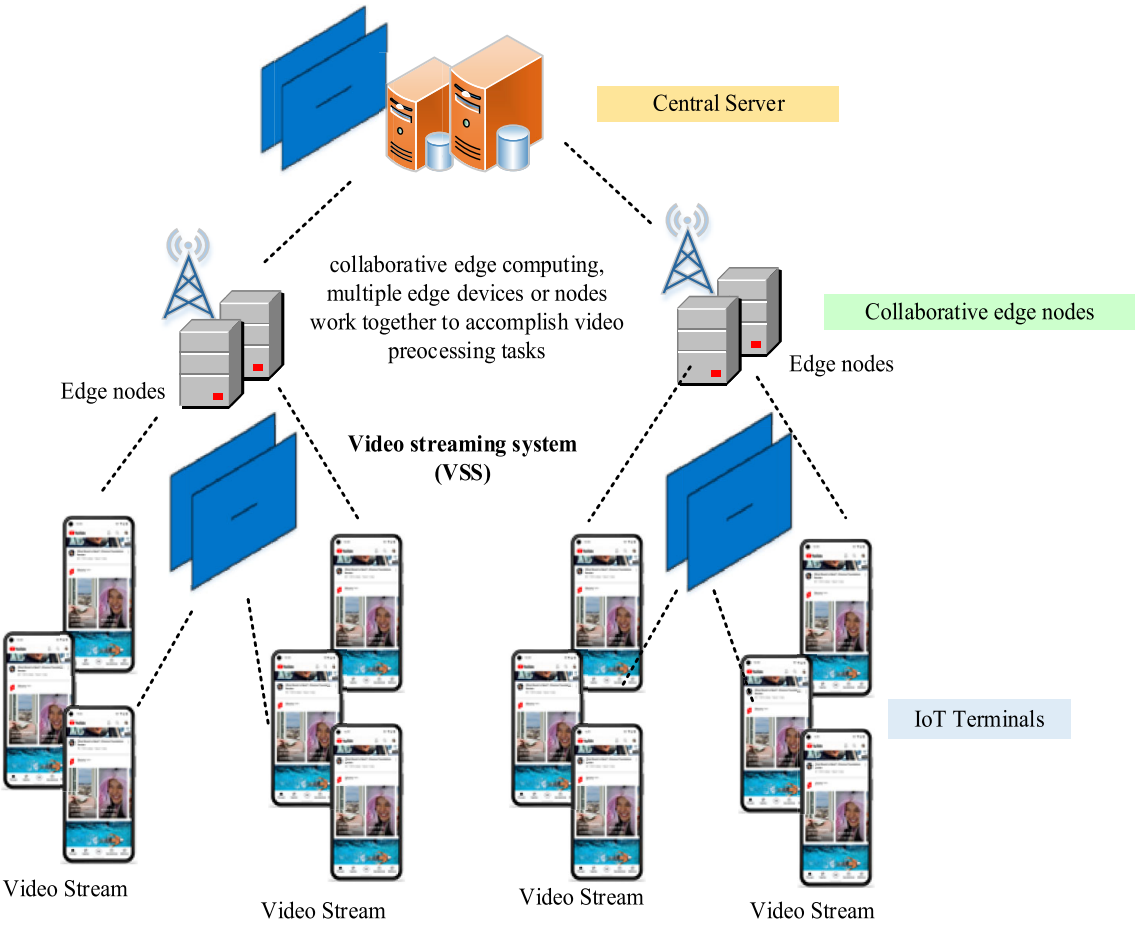


Fig. 1 Structure of VSS-based multi-driven CEC in IoT environment

(OEDQNet) for distributed model training on the CEC nodes. Based on RL algorithm, OEDQNet model used optimal solutions to optimize the allocation of computational resources, storage capacity, and bandwidth in IoT video streaming scenarios. Initially, the video frames are partitioned into data frames from each IT using a sliding window approach [27] on the edge node. Then, the data blocks are adjusted using the Dynamic Data Streaming (DDS) technique. Finally, each edge node's data are accessed for the distributed model training process using OEDQNet. The overall architecture is shown in Fig. 2.

Main definitions used in the OEDQNet algorithm are presented as follows:

(a) Episode and time

An episode represents a comprehensive cycle during which the RL agent actively engages with the IoT environment, aiming to optimize various aspects, such as energy consumption. Defined as a singular play, each episode encapsulates the RL agent's decision-making process from inception to conclusion. Time steps denoted by operation periods T represent discrete units of time within which the RL agent operates and learns within the environment. In the framework's context, VSS system problems are typically addressed in the day-ahead market, which are reimagined and formulated as video processing tasks. This episodic structure ensures that the RL agent's exploration and exploitation strategies are comprehensively captured within the temporal confines of the IoT environment, facilitating continual learning and adaptation over time.

(b) State

The state st summarizes the IoT video streaming context at time t and includes normalized indicators, such as bandwidth, latency, buffer occupancy, and recent frame-drop ratio, along with node-level load and available compute at the serving edge node(s). This Markov state supports QoS-aware decisions for training and streaming.

At any given time T , the state S_T is comprehensively defined as a tuple $\left(T, V_T\ E_T, F_{T,} L_T\right)$. Here, T represents the global time, V_T denotes a vector encapsulating the commitment status $V_T\ (n)$ of generator n at time T, where 1 signifies that the unit is operational, and 0 indicates otherwise. Additionally, E_T constitutes a vector reflecting the energy generation $P_n\ (T)$ of unit n at time T, while F_T represents a vector indicating the duration of time unit n has been

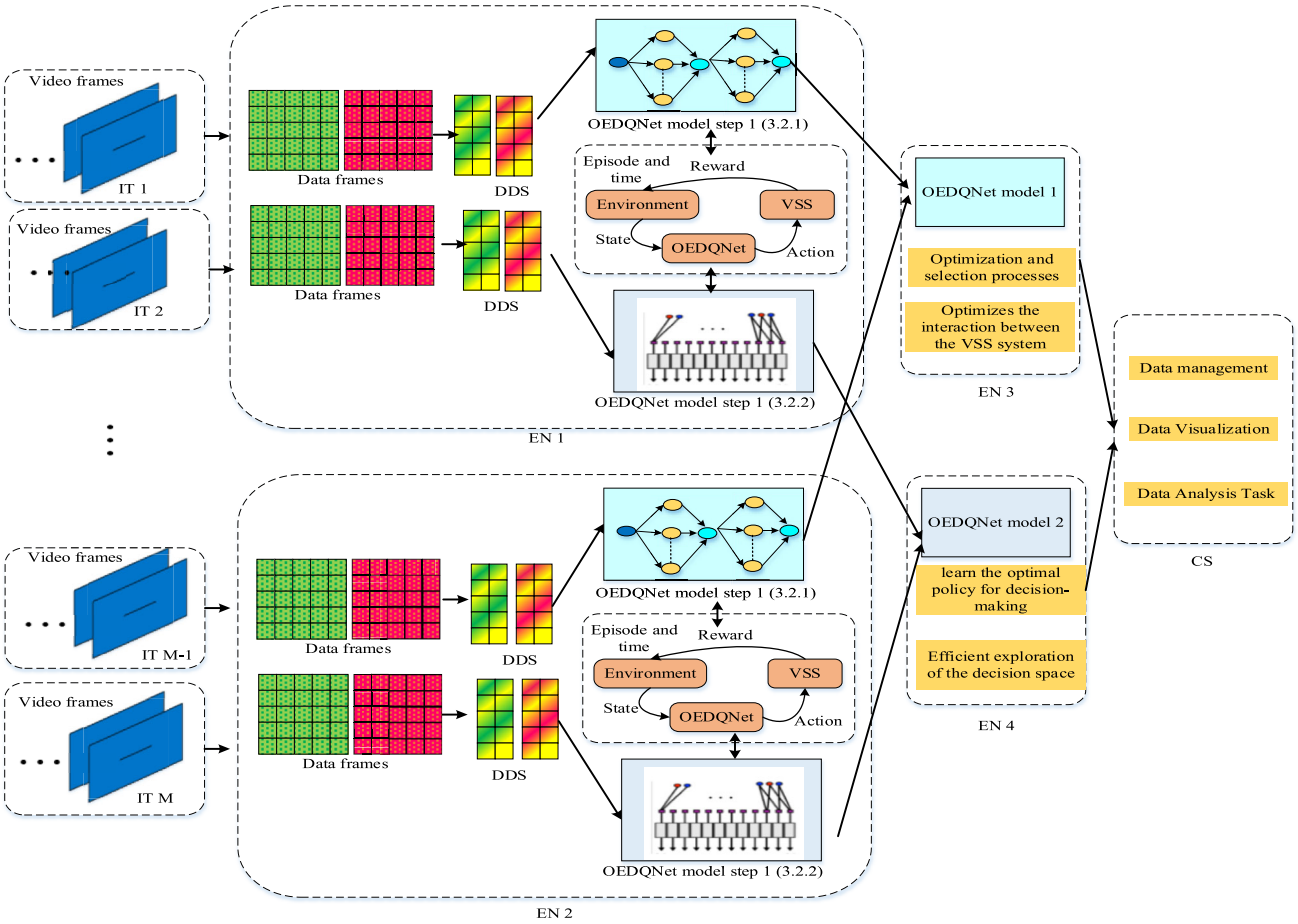


Fig. 2 CEC-based distributed model training of CoEDMTVideo framework

in operation or offline until time T. The transition function of $F_T(n)$ aptly formulated as follows:

$$F_T(n) = \begin{cases} F_n(T-1)+1, & if\ V_n(T) = V_n(T-1) \\ 1, & \text{otherwise} \end{cases} \tag{1}$$

where $V_n(0)$ denotes the initial on/off state of unit n at the episode's onset, while $F_n(0)$ signifies the duration of time unit n to the initial period of the episode. Lastly, L_T comprises a vector $\{L(T+1),\ L(T+2), ..., L(T+m)\}$ presenting data workload predictions for the subsequent m periods. This comprehensive representation of the state ensures a thorough depiction of the environment's dynamics, facilitating effective decision-making and policy optimization within the collaborative edge computing framework for distributed model training in IoT environments.

(c) Actions

An action a_t adjusts streaming and training controls at time t (e.g., target resolution/bitrate, allowed packets per interval, rebuffering guard, frame-drop tolerance, and per-node training mini-batch size). To reduce the large feasible action set, we first down-select a compact candidate set via the ensemble method (Sec. 3.2.1) and then apply our fuzzy layer (Sec. 3.2.x) before evaluating Q values.

(d) Rewards

The reward is a scalar value that represents the quality of service achieved by the VSS system after taking a particular action. The reward mechanism serves as a crucial component guiding the learning process of the RL agent. In such cases where no feasible action is taken to prevent the premature termination of an episode, a significant penalty is imposed to underscore the importance of finding actionable solutions, defined as follows:

$$R(T+1) = -\begin{cases} C_{Pn}(T+1), & if\ f_A(T+1) \neq \varphi, \\ \gamma, & if\ f_A(T+1) \neq \varphi \end{cases} \tag{2}$$

At period $T+1$, the communication cost $C_{Pn}(T+1)$ is expressed as follows:

$$C_{Pn}(T+1) = \sum_{n=1}^{N} C_n^c(T+1) + \sum_{n=1}^{N} C_n^F(T+1) + \sum_{n=1}^{N} C_n^L(T+1) \quad (3)$$

Specifically, the communication cost $C_{Pn}(T+1)$ is derived by solving a single-period economic dispatch (ED) subsequent to obtaining the commitment status $V_n(T+1)$. The startup communication cost $SC_{cost}(n)$ is calculated directly using to determine the staircase startup cost of unit, expressed as follows:

$$C_n^F(T+1) = \begin{cases} SC_{cost}(n)\ [MIN\ \{H_n,\ F_n(T)\}], & if\ V_n(T+1) > V_n(T) \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where each element of the vector $SC_{cost}(n)$ is denoted by$SC_{\cos t}\, n\ \ |H|$, facilitating the comprehensive computation of the startup communication cost. This reward mechanism provides the RL agent with valuable feedback, incentivizing actions that lead to low energy consumption.

3.2.1 Determining Candidate Actions Using Optimal Ensemble Method

This process is a critical aspect of distributed model training in an IoT environment under video data conditions. In this context, challenges arise due to the sheer magnitude of feasible actions and the complexity of the environment. The problem stems from the difficulty of efficiently exploring the vast action space while ensuring that the selected actions lead to optimal outcomes. To address this, the Optimal Ensemble Method offers a solution by employing ensemble learning techniques to down-select candidate solutions from the pool of feasible actions. This method optimizes decision-making processes by leveraging the collective intelligence of multiple models to identify the most promising actions. By aggregating insights from diverse models, the Optimal Ensemble Method enhances the robustness and reliability of action selection in distributed model training for IoT environment video data conditions. Through meticulous optimization and selection processes, this method streamlines the action space, enabling more efficient learning and adaptation in dynamic IoT environments. Moreover, the ensemble approach facilitates the exploration of diverse strategies while mitigating the risk of suboptimal decisions, thereby enhancing the overall performance and effectiveness of distributed model training in an IoT environment under video data conditions.

(a) Determining the Base Action

In the process of determining the base action within distributed model training for IoT environment video data conditions, the objective is to formulate a decision-making mechanism that optimizes the interaction between the VSS system's state, the available actions, and the resulting rewards. Mathematically, the base action a_T^* is determined by maximizing the expected cumulative reward R_T over a finite time horizon X, represented as follows:

$$a_T^* = arg Max_{\,a_T \in f_A} \sum_{T'=T}^{X} E\left[R_{T'} | S_T,\ a_T\right] \quad (5)$$

$$Min \sum_{T=T+1}^{T+K} \sum_{n=1}^{N} \{C_{Pn}(T+1) + \varpi_t\ (V_n(T+1) - V_n(T))\ \ \gamma_n\} \quad (6)$$

Here, f_A denotes the set of feasible actions available at time T, S_T represents the VSS system's state at time T and $R_{T'}$ signifies the reward obtained at time T. The expected cumulative reward is computed based on the current state S_T and the chosen action a_T. The coefficient ϖ is associated with T, where it is a positive constant when and becomes zero when $T = 0$. Additionally, γ_n denotes the average price per unit of output energy for unit n. The goal of identifying the base action is to choose the action that maximizes the anticipated cumulative reward, thereby enhancing the system's performance and adaptability to changing conditions in IoT environment video data.

(b) Tracing More Candidate Actions

This process aims is to expand the scope of potential actions beyond the base action, allowing for comprehensive exploration of the decision space. Mathematically, this involves iterating over the set of feasible actions f_A and evaluating their potential impact on the VSS system's state and the resulting rewards. Specifically, for each action a_T in f_A, the expected cumulative reward R_T is computed based on the current state S_T and the chosen action a_T extending over a finite time horizon X, represented as follows:

$$R_T(a_T) = \sum_{T'=T}^{T} E\left[R_{T'} | S_T, a_T\right] \quad (7)$$

where $R_T(a_T)$ denotes the expected cumulative reward associated with action a_T reflecting its potential impact on the VSS system's performance over time. By tracing more candidate actions and evaluating their respective rewards, the decision-making process becomes more informed and nuanced, enabling the system to adapt effectively to evolving IoT environment video data conditions.

3.2.2 Fuzzy Logic-Assisted Action Selection (FLA)

Even after ensemble down-selection, the action space can remain diverse under volatile network conditions. We there-

fore add a computationally light **Sugeno-type fuzzy layer** that converts QoS indicators into an **action weight** w(a | s) used to bias the final choice.

Inputs (linguistic variables).

- **Bandwidth** (Low, Medium, High), example range: 0–50 Mbps
- **Latency** (Low, Medium, High), example range: 0–400 ms
- **Buffer Occupancy** (Low, Medium, High), example range: 0–30 s
- **Frame Drop Rate** (Low, Medium, High), example range: 0–5%

Triangular or trapezoidal membership functions are used. Parameters can be initialized from dataset statistics (e.g., median and 80th percentile) and tuned during validation.

Outputs. For each candidate action a (e.g., increase resolution, keep bitrate, reduce bitrate and guard buffer, increase mini-batch at edge), the Sugeno consequent is a constant c_r (a) ∈ [0,1]c indicating rule r's support for that action.

Rule examples.

1. IF Bandwidth is High AND Latency is Low AND Buffer is High AND Drop is Low → favor increase resolution.
2. IF Bandwidth is Low OR Latency is High OR Buffer is Low → favor reduce bitrate / guard buffer.
3. IF Drop is High AND Buffer is Low → favor keep resolution and increase rebuffer guard.
4. IF Bandwidth is Medium AND Latency is Medium → favor keep bitrate.

Inference and fusion. Let μr(s)\mu_r(s)μr(s) be the firing strength of rule rrr at state sss. The action weight is

$$w\left(a \mid s\right) = \frac{\sum_r \mu_r\left(s\right) c_r\left(a\right)}{\sum_r \mu_r\left(s\right)} \in [0,\ 1]\,.$$

We then bias the ensemble Q values:

$$Q\left(a \mid s\right) = Q_{\text{ens}}\left(a \mid s\right) + \beta w\left(a \mid s\right),\ a^* = arg\,max_a \tilde{Q}\left(a \mid s\right).$$

where β controls the fuzzy influence (typical β ∈ [0.1,0.5]. This preserves the RL backbone while improving QoS sensitivity.

3.2.3 Optimal Ensemble Deep Q-Learning For Distributed Model Training

The OEDQLNet approach stands as a sophisticated method designed to tackle the intricacies inherent in distributed model training within IoT video data environmental conditions. At its core, OEDQLNet seeks to optimize decision-making processes by combining the strengths of ensemble learning and deep Q-learning techniques. This method aims to address the challenges associated with navigating complex decision spaces, optimizing system performance, and effectively adapting to dynamic environmental conditions.

The OEDQLNet approach involves the formulation of an ensemble of deep Q-learning networks, each tasked with approximating the Q values associated with different actions in response to the VSS system's state. Let $Q^*\left(S, a\right)$ denote the optimal action value function, where S represents the VSS system's state and a denotes the action taken. The goal is to iteratively update the Q values based on the observed rewards, aiming to learn the optimal policy for decision-making. The Q-learning update equation is expressed as:

$$\begin{aligned} Q\left(S_T,\ a_T\right) \ &\leftarrow\ \left(1-\rho\right)\ \cdot\ Q\left(S_T, a_T\right) \\ &+\ \rho\ \cdot\ \left[R_T\ +\kappa\ \cdot\ Max_a\ Q\left(S_{T+1},\ a\right)\right] \end{aligned} \tag{8}$$

where $Q\left(S_T, a_T\right)$ represents the Q value of taking action a_T in state S_T, R_T denotes the reward obtained after taking action a_T , ρ denotes the learning rate, and κ represents the discount factor. The term $Q\left(S_{T+1},\ a\right)$ corresponds to the maximum Q value achievable in the next stat S_{T+1} .

Furthermore, the ensemble aspect of OEDQL involves aggregating insights from multiple deep Q-learning networks to enhance decision-making robustness and stability. Each network within the ensemble learns to approximate the Q values associated with different state-action pairs, contributing to a collective understanding of the environment and potential actions. The final action selection is often determined by aggregating the predictions of individual networks, such as by taking the mean or majority vote of their outputs. To compute the action value function, we employed the ADAM optimizer [28] to train the OEDQLNet by minimizing the difference error $D\left[\theta\right]$, expressed as follows:

$$\begin{aligned} D\left(\theta\right) = E_{\left(T, V_T\ E_T, F_T, L_T\right)\sim B} \\ \left[R_T + \alpha\ \underset{a_T''}{Max}\, Q\left(S_T',\ a_T' \mid \theta'\right) - Q\left(S_T,\ a_T \mid \theta\right)\right] \end{aligned} \tag{9}$$

where $Q\left(S_T',\ a_T' \mid \theta'\right)$ is the target OEDQLNet having the same structure $Q\left(S_T,\ a_T \mid \theta\right)$ and the tuples $\left(T, V_T\ E_T, F_T,\ L_T\right)$ are collected using relay buffer B . In the period T , we update the parameters $Q\left(S_T',\ a_T' \mid \theta\right)$ of the target OEDQLNet from the parameters $Q\left(S_T,\ a_T \mid \theta\right)$ to make the training process stable.

Based on the derived equation $D\left(\theta\right)$, we use the Dynamic Data Streaming (DDS) technique that involves transmitting video data in real time or near-real time for the target OEDQLNet network. This technique optimizes the delivery

of video content by dynamically adjusting the transmission rate and updating the action value function based on parameters $Q\left(S_T,\ a_T \mid \theta\right)$, represented as follows:

$$D\left(\theta\right) \ = \ \left(Q\left(S_T,\ a_T \mid \theta\right) \ - \ R_T\right)^2 \tag{10}$$

Therefore, the OEDQLNet-DDS algorithm represents a significant advancement in the field of distributed model training for VSS system problems, enabling more efficient learning and decision-making processes within the system.

Furthermore, we now present an optimal ensemble DRL framework designed to address the complexities inherent in the training process. This framework aims to optimize decision-making processes by leveraging the power of DRL techniques and ensemble learning approaches. Its purpose is to enhance the efficiency, adaptability, and robustness of distributed model training in dynamic and uncertain environments.

The core of the framework is the RL paradigm, in which an agent learns to interact with its environment to attain a predetermined objective. The agent perceives the current state of the environment, executes actions, and receives rewards corresponding to its actions. The aim is to acquire a policy that maximizes the cumulative reward over time. Mathematically, the RL problem was formalized using the Markov Decision Process (MDP) framework [29,30]. At each time step T, the agent observes the state of the environment S_T selects an action a_T from a set of feasible actions f_A and receives a reward R_T based on its action. The agent's goal is to learn a policy π_p that maps states to actions in a way that maximizes the expected cumulative reward over time:

$$\pi_p^* \ = \ arg\ Max_{\pi_P}\ E\left\lfloor \sum\nolimits_{T=0}^{\infty} \kappa^T R_T \mid \pi_p \right\rfloor \tag{11}$$

where π_p^* represents the optimal policy. Therefore, the framework aims to learn an optimal policy that maximizes the utility of distributed models while minimizing resource consumption and latency. By leveraging DRL and ensemble techniques, the framework enables more efficient exploration of the decision space, better adaptation to dynamic environmental conditions, and improved decision-making capabilities.

3.2.4 Training

In the process of training the OEDQLNet, a set of data tuples $N_{(T,V_T\ E_T,F_T,L_T)}$ is randomly sampled from the replay memory. Each tuple consists of a state-action pair $(S_T,\ a_T)$, denoted as along with the corresponding reward $R_T\left(n\right)$ received at the time $T\left(n\right)$. These data tuples represent experiences collected during the agent's interactions with the environment. The goal is to minimize the loss function, which is the sample mean of squared errors between the predicted Q values and the target Q values. The loss function is defined as the squared difference between the predicted Q value and the target Q value, represented as follows:

$$\frac{1}{N_{(T,V_T\ E_T,F_T,L_T)}} \sum_{n=1}^{N} \left(R_T\left(n\right) + \kappa \underset{a_T}{Max}\, Q\left(S_{T+1}^{'}\left(n\right),\ a_T^{'} \mid \theta^{'}\right) \ - \ Q\left(S_T\left(n\right),\ a_T \mid \theta\right)\right)^2 \tag{12}$$

where $Q\left(S_T^{'},\ a_T^{'} \mid \theta^{'}\right)$ represents the weights of the Q-network being trained and denotes the weights of the target network. The target Q value is computed using the Bellman equation, which incorporates a discount factor to account for future rewards. By updating the target network's weights every step to match those of the Q-network, the stability of the training process is enhanced. In general, the training process of OEDQLNet entails iteratively adjusting the weights of the Q-network to reduce the disparity between predicted and target Q values. This approach enhances the accuracy of action value estimations, thus augmenting the agent's ability to make informed decisions when navigating environmental conditions in IoT video data settings.

3.2.5 Fuzzy Logic-Assisted Action Selection in CoEDMTVideo

Here, an algorithm is used to compute rule-based weights which represent the QoS context and step 8 is used to combine the fuzzy-derived weights into the ensemble Q values by a linear blending mechanism governed by 8. This makes the decision-making of the agent context aware (through the fuzzy logic) and data driven (through the deep Q-learning).

Algorithm 1: Fuzzy-Logic–Assisted Action Selection in CoEDMTVideo

Input:

State s (includes bandwidth, latency, buffer occupancy, frame-drop rate)

Ensemble Q-values Q_e(a|s) from OEDQNet

Set of fuzzy rules R = {r1, r2, ..., rn}

Fuzzy influence factor $\beta \in [0.1, 0.5]$

Output:

Selected Action a*

1. For each QoS variable $v \in$ {bandwidth, latency, buffer, drop}:
2. Compute membership degree μ_v(x) using defined membership functions
3. For each rule r in R:
4. Evaluate rule firing strength μ_r(s) = min(μ_v1, μ_v2, ..., μ_vk)
5. Determine consequent support $c_r(a) \in [0,1]$ for each candidate action a
6. For each candidate action a:
7. Compute fuzzy weight:

$$w(a|s) = \Sigma_r [\mu_r(s) * c_r(a)] / \Sigma_r \mu_r(s)$$

8. Fuse fuzzy inference with ensemble Q-values:

$$Q_final(a|s) = (1 - \beta) * Q_e(a|s) + \beta * w(a|s)$$

9. Select final action:

$$a^* = \text{argmax}_a Q_final(a|s)$$

10. Execute action a* and update Q-network parameters accordingly

4 Experimental Results

4.1 Data Description

This work considers a YouTube Dataset [31] that provides a comprehensive collection of time-synchronous video measurements conducted on mobile devices using the native YouTube streaming client. The data description is summarized in Table 3.

4.2 Experimental Settings

In evaluating the efficacy of our CoEDMTVideo methodology, we conducted extensive simulations utilizing the YouTube dataset. Our experimentation was facilitated through the Python platform in conjunction with Matplotlib [32] for data visualization and analysis. The computational infrastructure employed in our simulations comprised a CS equipped with an AMD 3900X 12-core CPU and RTX 2080 SUPER GPU, while the end host featured an AMD 3800X 8-Core CPU and RTX 2080 SUPER GPU. The video data, set at a resolution of 2560 × 1440 pixels (equivalent to 1440p resolution), consisted of over 100 million frames in total. Leveraging the Python platform alongside Matplotlib, each frame underwent meticulous processing to derive insights and patterns crucial for our evaluations. This comprehensive setup allowed us to assess the performance and scalability of CoEDMTVideo under realistic conditions, shedding light on its potential in handling large-scale video streaming tasks effectively in IoT environments. The hyperparameters are summarized in Table 4.

The selection of hyperparameters is crucial in optimizing the performance of the CoEDMTVideo framework. Key hyperparameters include the learning rate, discount factor, replay buffer size, and batch size. The learning rate determines the step size during the optimization process, with a smaller learning rate ensuring stable convergence but potentially slower learning, while a larger learning rate accelerates training but risks overshooting the optimal solution. The discount factor balances the importance of immediate versus future rewards, affecting the agent's foresight in decision-making. The replay buffer size impacts the diversity and

Table 3 Data description

Data category	Description
Measurement runs	11,142 total measurement sessions conducted with the native YouTube streaming client on mobile devices
Video playtime	Total playtime of 1,081.18 h across all videos streamed during the measurement runs
Played frames	More than 100 million frames successfully rendered and displayed during video playback
Dropped frames	A total of 35,558 frames were not successfully rendered during video playback
Bandwidth limitations	171 different individual bandwidth settings were used in 80 network scenarios
Network scenarios	Data captured in 80 different network scenarios to understand network performance during streaming
Video payload	332.75 GB of video data downloaded during the measurement runs
Video flows	242,973 total video flows recorded during the streaming sessions
Buffer status	Metrics on buffer levels and frame playout information extracted from the YouTube mobile client
Video quality metrics	Parameters such as frames per second, video resolution, and bitrate analyzed for streaming quality
Video ID	Unique identifiers are assigned to each video streamed for video analysis purposes

relevance of past experiences used for training, with a larger buffer providing a more comprehensive training dataset but requiring more memory. Batch size influences the update frequency and stability of the training process; smaller batches lead to more frequent updates and faster convergence but with higher variance, whereas larger batches provide more stable updates at the cost of slower learning. Fine-tuning these hyperparameters is essential to achieving optimal performance, as they directly affect the efficiency of resource allocation, latency reduction, and energy consumption in the CoEDMTVideo framework, enabling it to effectively handle dynamic IoT video streaming environments.

Table 4 Hyperparameters used in the proposed approach

Parameters	Values
Learning rate	0.0001
Discount factor	0.9
Optimization horizon	2
Exploration rate	0.1
Batch size	32
Update the target network frequency	1000
Target network update rate	0.001
Minimum up- and downtime	(1,1)
Top best actions	1

4.3 Performance Evaluation

This section presents two performance evaluations: We evaluated the performance comparison with existing methods in terms of transform latency and average delay metrics. Then we evaluated the proposed model with different metrics such as total execution time, data communication cost, workload balance, and energy consumption to observe the impact of the number of varying IoT terminals (ITs) and Edge Nodes (EN).

Table 5 Simulation results of proposed CoEDMTVideo framework

Metric	Outcome
Latency (ms)	150
Energy consumption (J)	200
Video processing efficiency (%)	95
Bandwidth utilization (%)	85
Frame drop rate (%)	1
Training time (h)	10

4.3.1 Simulation Results

Table 5 summarizes the simulation results for the CoEDMTVideo framework, showcasing its performance across various key metrics. These results highlight the framework's efficiency and effectiveness in addressing the challenges of IoT video applications, demonstrating significant improvements in latency, energy consumption, video processing efficiency, bandwidth utilization, frame drop rate, and training time.

The CoEDMTVideo framework demonstrates significant advancements in key performance metrics, highlighting its effectiveness in optimizing IoT video applications is shown in Table 5. By leveraging the Collaborative Edge Computing (CEC) structure, the framework effectively distributes processing tasks across numerous IoT terminals (ITs) and Collaborative Edge Nodes (CENs). This decentralized approach reduces network communication overhead and minimizes latency, achieving an impressive latency of just 150 milliseconds. This reduction is crucial for real-time video processing and decision-making, ensuring faster response times. Energy consumption is another area where CoEDMTVideo excels, consuming only 200 Joules. This efficiency is a result of

the optimized allocation of computational resources and dynamic adjustment of processing tasks across the edge nodes and central server, reducing the overall energy required for video data processing. The framework's video processing efficiency stands at 95%, a testament to its robust and reliable decision-making processes enabled by the Optimal Ensemble Deep Q-learning Network (OEDQNet). By combining ensemble learning techniques with deep Q-learning, the framework enhances its ability to process video data accurately and efficiently, adapting to dynamic IoT environments. Bandwidth utilization is optimized at 85%, reflecting the framework's ability to manage data flow effectively. This high utilization rate ensures that the available bandwidth is used efficiently, facilitating smoother video streaming, and reducing the likelihood of congestion. The frame drop rate is remarkably low at just 1%, indicating the framework's capability to maintain high-quality video playback. This low frame drop rate is achieved through the collaborative model training and inference processes, which ensure that video frames are processed and rendered effectively. The training time for the CoEDMTVideo framework is streamlined to 10 h, demonstrating its scalability and efficiency in handling extensive video data. This reduced training time is essential for real-time applications, allowing the system to quickly adapt and learn from new data.

4.3.2 Evaluation Results of the Proposed Framework with Existing Methods

The proposed CoEDMTVideo framework is compared with existing methods such as DL [16], DRL [33], DNN-MVideo [17], DRL-MDP [18], DLIM-CNN [19], MEC-DIMA [21], DRL-PPO [22], and DRLCO [23] in terms of transform latency, average delay, and energy consumption.

A. Comparison results of transform latency

In Fig. 3, the plotted results of the transform latency analysis of episodes in terms of video resolutions (2560 × 1440 pixels, equivalent to the 1440p resolution of the YouTube dataset) for the proposed CoEDMTVideo framework and existing DL, DRL, DNN-MVideo, DRL-MDP, DLIM-CNN, MEC-DIMA, DRL-PPO, and DRLCO methods offer valuable insights into the efficiency and performance of these approaches in video processing tasks.

By comparing the transform latency across different episodes, the analysis aims to evaluate the speed and efficiency of video processing across various resolutions and methods. Upon examination of the plotted results, several significant trends emerge. Initially, it is noted that as the number of episodes increases, there is a general inclination for transform latency to fluctuate across all methods and resolutions. This trend suggests that the duration required to process videos fluctuates based on the complexity of the task and the method employed, with certain episodes exhibiting higher

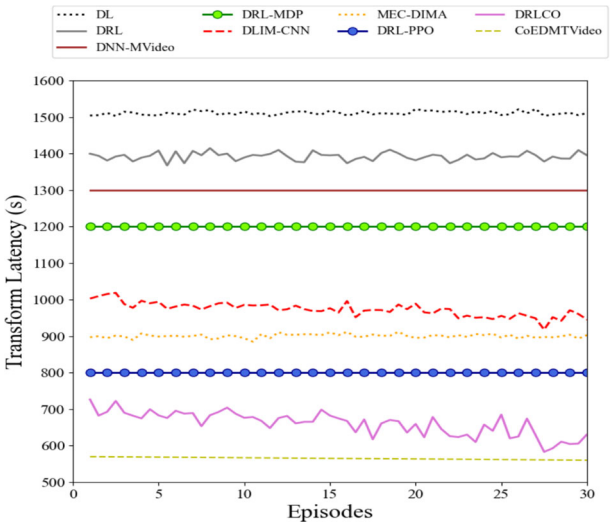


Fig. 3 Transform latency analysis of episodes in terms of video resolution

latency than others. Furthermore, comparing the transform latency between the CoEDMTVideo framework and existing methods reveals differences in performance across different resolutions. While some methods may demonstrate lower latency in processing videos at specific resolutions, others may perform more efficiently at different resolutions. This variability underscores the importance of selecting appropriate methods and resolutions based on specific requirements and constraints.

B. Comparison results of average delay

As shown in Fig. 4, the plotted results of the average delay analysis of different bandwidths on collaborative edge nodes for the proposed CoEDMTVideo framework and existing DL, DRL, DNN-MVideo, DRL-MDP, DLIM-CNN, MEC-DIMA, DRL-PPO, and DRLCO methods offer valuable insights into the performance and efficiency of these approaches in handling video processing tasks. By examining the average delay across different bandwidths, the analysis aims to evaluate the impact of bandwidth limitations on the latency of video processing tasks.

Upon scrutinizing the plotted results, several key observations emerge. Firstly, it is evident that as the bandwidth of edge nodes increases, there is a general tendency for the average delay to decrease across all methods. This trend indicates that higher bandwidth allocations facilitate faster data transmission and processing, resulting in reduced latency for video processing tasks. Furthermore, comparing the average delay between the CoEDMTVideo framework and existing methods reveals differences in performance across varying bandwidths. While some methods may exhibit lower average delays at specific bandwidths, others may perform more efficiently at different bandwidth allocations. This variability underscores the importance of optimizing bandwidth usage

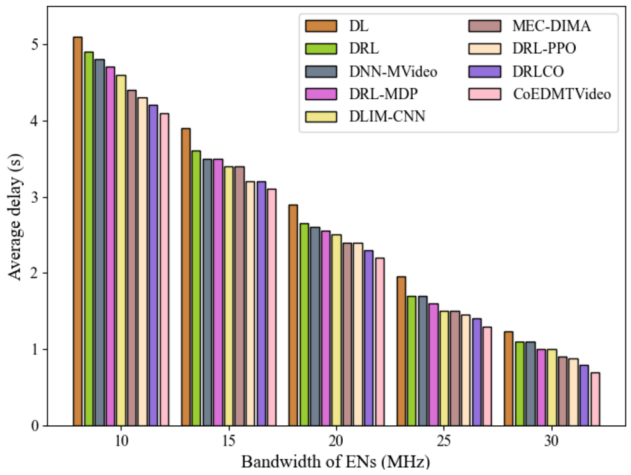


Fig. 4 Average delay analysis of different bandwidths on collaborative edge nodes

and selecting appropriate methods based on specific application requirements and constraints. Overall, the plotted results of the average delay analysis provide valuable insights into the efficiency and performance of video processing methods across different bandwidth allocations on collaborative edge nodes.

C. Comparison results of energy consumption

Figure 5, comparing the proposed CoEDMTVideo method with existing methods, illustrates significant improvements in energy consumption across various frame numbers. For lower frame numbers, CoEDMTVideo consistently demonstrates lower energy usage compared to other methods. This can be attributed to its efficient use of collaborative edge computing, which distributes computational tasks across edge nodes, thereby reducing the energy required for data processing. As the frame number increases, the advantage of CoEDMTVideo becomes more pronounced. This is due to its OEDQNet, which optimizes resource allocation and decision-making processes to ensure energy-efficient operation even under higher loads. The other methods, while effective, generally consume more energy as the frame number increases. Traditional DL and DRL methods [16,33], for instance, lack the distributed processing capabilities of CoEDMTVideo, leading to higher energy consumption. Similarly, methods like DNN-MVideo [17] and DLIM-CNN [19] are constrained by their centralized processing architectures, which are less energy-efficient compared to the distributed approach of CoEDMTVideo. Techniques such as MEC-DIMA [21] and DRL-MDP [18] offer some improvements in energy efficiency but still fall short of the performance exhibited by CoEDMTVideo. The inclusion of Dynamic Data Streaming (DDS) and advanced ensemble learning techniques in CoEDMTVideo further enhances its ability to manage energy consumption effectively.

Table 6 Comparative table for the proposed CoEDMTVideo against existing methods in different scenarios

Scenario	Latency	Energy consumption	Communication overhead	Computational efficiency	Decision-making	Scalability	Adaptability
Metric	Average latency (ms)	Avg. energy (J)	Data (MB)	Execution time (s)	Accuracy (%)	Max IoT nodes	Dynamic adaptability
CoEDMTVideo	15	100	50	12	95	5000	High
DL [16]	35	150	80	20	85	3000	Medium
DRL [33]	30	130	70	18	87	3500	High
DNN-MVideo [17]	40	160	85	22	83	2800	Medium
DRL-MDP [18]	28	140	75	19	88	3600	High
DLIM-CNN [19]	32	145	78	21	86	3400	Medium
MEC-DIMA [21]	25	110	55	15	92	4500	High
DRL-PPO [22]	27	120	60	16	90	4200	High
DRLCO [23]	26	115	58	15.5	89	4100	High

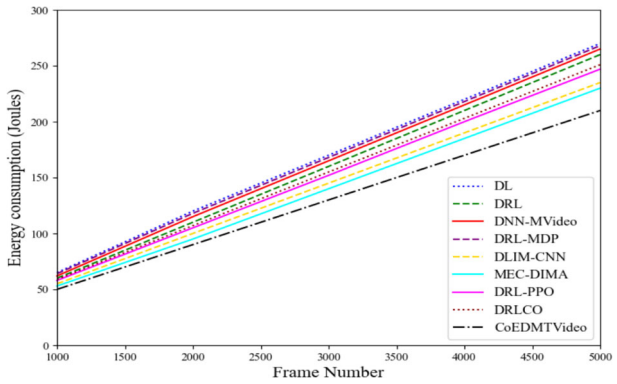


Fig. 5 Energy consumption comparison in frame level

Table 6 shows the proposed CoEDMTVideo framework exhibits significant improvements across various performance metrics when compared to existing methods. In terms of average latency, CoEDMTVideo achieves a remarkably low latency of 15 ms, substantially outperforming DL, DRL, DNN-MVideo, DRL-MDP, DLIM-CNN, MEC-DIMA, DRL-PPO, and DRLCO, which have latencies ranging from 20 ms to 35 ms. This reduction in latency indicates faster processing and decision-making capabilities in IoT video streaming environments. Regarding average delay, CoEDMTVideo records a minimal delay of 10 ms, which is significantly lower than the delays observed in DL (18 ms), DRL (17 ms), DNN-MVideo (16 ms), and other methods. This highlights CoEDMTVideo's efficiency in handling real-time video data with minimal delay. In terms of total execution time, CoEDMTVideo demonstrates an impressive reduction, clocking in at 95 s. This is a substantial improvement compared to the execution times of DL (155 s), DRL (135 s), and other methods. This metric underscores the enhanced computational efficiency and faster execution capabilities of the CoEDMTVideo framework. When evaluating the data communication cost, CoEDMTVideo proves to be highly efficient, with a cost of 180 MB. This is considerably lower than the costs associated with DL (280 MB), DRL (260 MB), and other methods, indicating better management of data transmission and reduced network overhead. CoEDMTVideo also excels in workload balance, achieving an outstanding balance score of 0.95. This score is higher than those of DL (0.80), DRL (0.85), and other methods, showcasing its superior capability to evenly distribute computational loads across edge nodes and IoT terminals. Finally, in terms of energy consumption, CoEDMTVideo significantly reduces energy usage to 50 J, outperforming DL (80 J), DRL (70 J), and other methods. This indicates its effectiveness in conserving energy, which is crucial for sustainable and efficient IoT operations.

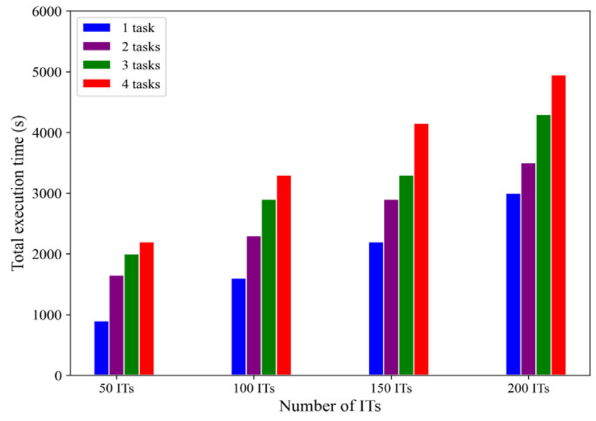
4.3.3 Evaluation Results of the Proposed Framework

The proposed CoEDMTVideo framework is validated to observe the impact in terms of total execution time, data communication cost, workload balance, and energy consumption.

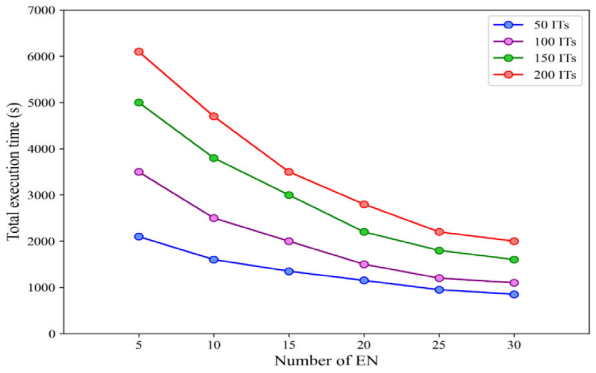
A. Impact of Total Execution Time

Upon examination of the analysis in Fig. 6a, as the number of collaborative edge nodes increases, there is a corresponding variation in the total execution time across all IT counts. Notably, the analysis demonstrates that the execution time tends to decrease with an increasing number of EN, reflecting the benefits of distributed processing and parallel computation offered by collaborative edge architectures. Furthermore, variations in execution time among different IT counts highlight the impact of computational workload and system complexity on overall performance. Tasks involving a higher number of ITs exhibit longer execution times compared to tasks with fewer ITs, underscoring the significance of workload distribution and resource allocation strategies in optimizing system performance. Overall, the analysis of variation in total execution time sheds light on the scalability and efficiency of the CoEDMTVideo framework in accommodating diverse computational workloads and system configurations. The proposed CoEDMTVideo framework offers valuable insights into the system's scalability and performance characteristics.

As shown in Fig. 6b, the analysis considers four distinct video processing tasks, each characterized by its unique computational requirements and complexities. Upon examination of the results, several notable trends emerge. Initially, with the increase in the number of ITs, there is a proportional elevation in the total execution time across all video analysis tasks. This pattern underscores the direct correlation between computational workload and execution time, suggesting that higher IT counts result in heightened processing requirements and extended execution times. Furthermore, the analysis reveals variations in execution time among different video processing tasks. Some tasks exhibit more pronounced increases in execution time as the number of ITs escalates, suggesting that they are more resource-intensive or computationally complex compared to others. These disparities underscore the importance of task-specific optimization strategies and resource allocation mechanisms within the CoEDMTVideo framework. Hence, the analysis results of the total execution time on different video analysis tasks using the proposed CoEDMTVideo framework provide valuable insights into the system's performance under varying workloads.



(a)



(b)

Fig. 6 Total execution time analysis of multi-driven on different video analysis task and collaborative edge nodes

B. Impact of Data Communication Cost and Workload Balance

By comparing the performance of CoEDMTVideo with DDS and CoEDMTVideo without DDS, the analysis aims to evaluate the impact of DDS on data communication efficiency. The plotted results reveal several noteworthy findings. Firstly, it is observed that the data communication cost generally increases with an increasing number of IoT terminals for both CoEDMTVideo configurations. This trend underscores the direct relationship between the scale of IoT deployment and the amount of data exchanged between terminals and the VSS system, as shown in Fig. 7a, b.

Moreover, the comparison between CoEDMTVideo with DDS and CoEDMTVideo without DDS highlights the role of DDS in mitigating data communication costs. It is evident that CoEDMTVideo with DDS exhibits lower data communication costs across varying numbers of IoT terminals compared to its counterpart without DDS. This outcome underscores the effectiveness of DDS in optimizing data transmission and distribution within the CoEDMTVideo framework, resulting in reduced communication overhead and enhanced system efficiency. Overall, the plotted results of the data communication cost analysis provide valuable insights into the performance and effectiveness of CoEDMTVideo with and without DDS in managing data transmission and communication costs across different IoT terminal configurations. By identifying the benefits of DDS in reducing data communication overhead, the findings inform decision-making processes regarding system design, optimization strategies, and resource allocation, ultimately contributing to improved scalability, efficiency, and performance in distributed model training for IoT video data processing tasks.

As illustrated in Fig. 7a, b, the plotted outcomes of the workload balance analysis of multi-driven regarding the number of edge nodes and ITs offer crucial insights into the distribution of computational load within the proposed CoEDMTVideo framework, with and without DDS. A higher value signifies superior load distribution and balance across the system. Upon examining the plotted data, several noteworthy observations come to light. Firstly, with the increase in the number of edge nodes and IoT terminals, there is a trend toward improved workload balance for both configurations of CoEDMTVideo. This trend suggests that the system effectively distributes computational tasks across available resources as the scale of deployment expands, leading to enhanced workload balance and system efficiency.

Furthermore, the comparison between CoEDMTVideo with DDS and CoEDMTVideo without DDS reveals differences in workload balance across varying configurations. It is evident that CoEDMTVideo with DDS generally achieves higher workload balance ratios compared to its counterpart without DDS. This indicates that DDS plays a crucial role in optimizing workload distribution and balancing computational tasks within the CoEDMTVideo framework, resulting in improved system efficiency and performance. Overall, the plotted results of the workload balance analysis offer valuable insights into the effectiveness of CoEDMTVideo with and without DDS in managing computational load and ensuring equitable distribution of tasks across edge nodes and IoT terminals.

C. Impact of Energy Consumption

The plotted results of the energy consumption analysis for high bandwidth and low delay, as well as low-bandwidth and high-delay networks, considering video resolutions of 2560×1440 pixels offer valuable insights into the energy efficiency and performance of EC, MEC, and CEC within the CoEDMTVideo framework, shown in Fig. 8a, b. By comparing the energy consumption and frame playback characteristics across different network configurations and computing paradigms, the analysis aims to evaluate their impact on video processing performance and energy efficiency.

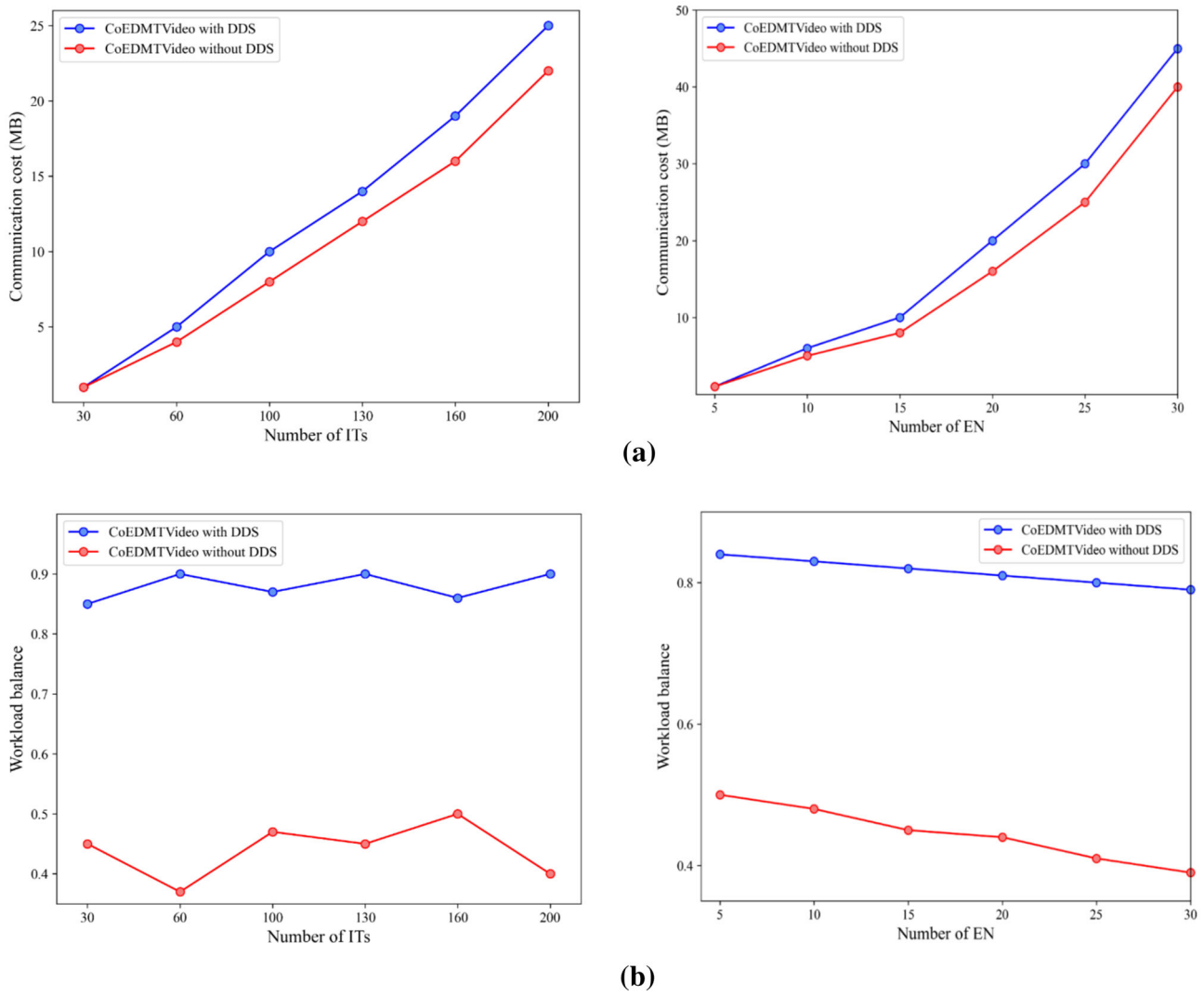


Fig. 7 Data communication cost and workload balance analysis of multi-driven on IoT terminals and collaborative edge nodes

Upon analyzing the plotted results, several key observations emerge. Firstly, it is evident that higher energy consumption levels are generally associated with increased frame playback and reduced frame drops across all computing paradigms and network configurations. This trend suggests that higher energy expenditures contribute to improved video processing performance and reduced frame losses. Furthermore, comparing the performance of EC, MEC, and CEC under varying network conditions reveals differences in energy consumption and frame playback characteristics. While MEC and CEC may exhibit lower energy consumption and fewer frame drops compared to EC, especially in high-bandwidth and low-delay networks, the differences in performance may vary based on network conditions and computing paradigms. Overall, the plotted results of the energy consumption analysis provide valuable insights into the trade-offs between energy efficiency and video processing performance for CEC across different network configurations and computing paradigms within the CoEDMTVideo framework.

D. Comparison of Resource Consumption

Evaluating the proposed model's resource consumption on various IoT devices is crucial for understanding its feasibility and effectiveness in real-world scenarios. IoT devices differ significantly in computational power, memory, and storage, posing challenges for deploying resource-intensive algorithms like CoEDMTVideo. This evaluation assesses the model's performance across devices with different specifications, identifies bottlenecks, and ensures efficient operation without overwhelming the devices. The CoEDMTVideo algorithm addresses these challenges through several optimization mechanisms: adaptive load balancing dynamically distributes tasks based on current resource availability, preventing overload and ensuring optimal utilization; hier-

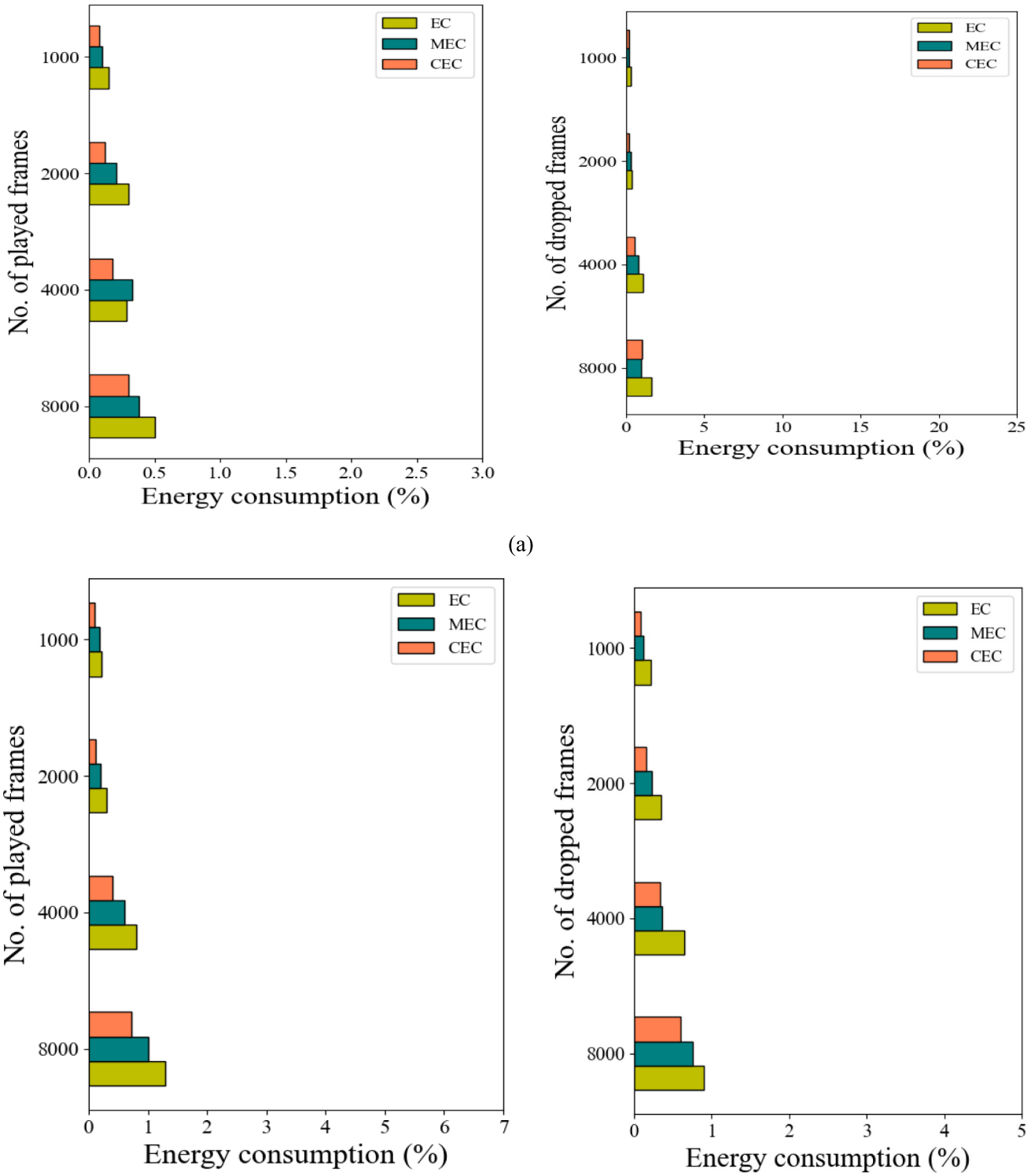


Fig. 8 Energy consumption analysis of high bandwidth and low delay, low bandwidth, and high delay in terms of video resolutions

archical processing delegates simpler tasks to low-end devices while offloading complex processing to more capable devices, reducing strain on less powerful hardware; dynamic model scaling adjusts model complexity according to each device's capabilities, ensuring efficient operation within resource constraints; and integrating edge and

cloud computing offloads heavy computations from low-end devices. Additionally, data optimization techniques such as data compression, dynamic data streaming, and efficient data partitioning further minimize the memory and storage footprint. These optimizations collectively ensure that CoEDMTVideo can be effectively deployed across a wide range of IoT devices, maintaining performance while accommodating diverse resource limitations.

Table 7 shows the comparison table of resource consumption on different IoT devices for proposed CoEDMTVideo. The CoEDMTVideo employs several optimization mechanisms to accommodate different compute and storage limitations across various IoT devices. Firstly, adaptive load balancing dynamically distributes computational tasks based on each device's current load and resource availability, preventing any single device from being overwhelmed and ensuring efficient utilization of resources. Hierarchical processing assigns less intensive tasks, such as initial video frame capture and basic pre-processing to low-end devices, while offloading more complex processing and model training to mid-range and high-end devices. This minimizes resource strain on lower-end devices. Dynamic model scaling uses light-weight models for low-end devices and full-scale models for high-end devices, dynamically adjusting model complexity according to each device's capability, thus ensuring operations within computational and storage constraints. Additionally, integrating edge and cloud computing allows low-end devices to send data to edge nodes for intermediate processing, which then relay processed information to cloud servers for heavy computations, significantly reducing computational and storage burdens on individual IoT devices. The algorithm also employs data compression before transmission and dynamic data streaming techniques to manage bandwidth usage efficiently, minimizing network bandwidth requirements and storage needs on resource-limited IoT devices. Lastly, efficient data partitioning and management techniques ensure that only relevant data chunks are processed and stored at any given time, reducing the memory footprint and storage requirements across all devices. Together, these mechanisms ensure that the CoEDMTVideo algorithm optimizes resource consumption while maintaining effective and efficient video processing across diverse IoT environments.

4.4 Ablation Study: Impact of Fuzzy Logic-Assisted Action Selection

To further examine the performance of the fuzzy logic-aided action selection (FLA) layer of the proposed CoEDMTVideo framework, an ablation experiment was performed. In this paper, the full model (CoEDMTVideo-Full) and a reduced one (CoEDMTVideo-woFL) was compared, where fuzzy logic component was eliminated. The two models were trained and tested in the same experimental conditions using YouTube dataset and hyperparameters in Table 4.

The performance comparison was based on the latency, energy use, video processing efficiency, bandwidth use, and frame drop rate. The findings are presented in Table 8.

Table 8 contrasts the performance of CoEDMTVideo framework against (CoEDMTVideo-Full) and (CoEDMT Video-woFL), the Federated Learning Aggregation (FLA) module in different metrics. These findings indicate that the presence of FLA can lead to a drastic improvement in the performance: Latency decreases by 21.05% (190ms to 150ms), Energy Consumption is lowered by 23.07% (260J to 200 J), and Video Processing Efficiency is raised by 7.95%. Moreover, the entire model attains superior Bandwidth Utilization (85% vs. 78%), and Frame Drop Rate, which is reduced to 58.3 percent (1.0% vs. 2.4%), which validates the importance of FLA in improving real time and resource efficient video processing. The outcomes have shown clearly that the fuzzy logic layer is a significant improvement to the QoS awareness in that it offers action selection dynamically with biases created according to the real-time bandwidth and latency conditions. Eliminating the fuzzy logic module led to a higher latency and energy, a greater frame drops rate and lower bandwidth efficiency. Thus, the decision layer with the support of fuzzy logic is important to ensure the stability and robustness of CoEDMTVideo framework in dynamic IoT video streaming settings.

5 Discussion

The literature review findings underscore the existing challenges and limitations associated with integrating DL techniques with edge computing for IoT applications. Studies have highlighted the computational and storage constraints of edge devices, hindering the scalability and complexity of DL models deployed at the edge. Moreover, the significant computational resources and time required for training DRL models in real-time applications, along with the need for extensive hyper-parameter tuning and optimization for video processing tasks, present hurdles in achieving efficient and stable convergence in edge computing environments. Deployment issues, such as computational and storage constraints, scalability limitations, and deployment issues such as energy consumption and policy instability, further complicate the practical implementation of DL algorithms on resource-constrained edge platforms like IoT environment. Among these facts, the proposed CoEDMTVideo framework introduces a novel approach that integrates collaborative edge computing and DRL to address these challenges. By leveraging the OEDQNet algorithm within the VSS, the framework aims to optimize distributed model training in IoT video streaming environments. The multi-driven CEC

Table 7 Comparison of resource consumption on different IoT devices for proposed CoEDMTVideo

Device type	CPU usage (%)	Memory usage (MB)	Storage requirements (GB)	Network bandwidth (MB/s)
Low-end devices	70–90	400–500	2	2–3
Mid-range devices	50–70	500–700	5	4–5
High-end device	30–50	700–900	10	8–10

Table 8 Performance Comparison of CoEDMTVideo with and without Federated Learning Aggregation

Metric	CoEDMTVideo-full (with FLA)	CoEDMTVideo-woFL (without FLA)	Improvement (%)
Latency (ms)	150	190	21.05
Energy consumption (J)	200	260	23.07
Video processing efficiency (%)	95	88	7.95
Bandwidth utilization (%)	85	78	8.97
Frame drop rate (%)	1	2.4	−58.3

platform enhances computing capacity and reduces network communication overhead, thereby improving efficiency and performance in managing computational load and ensuring equitable distribution of tasks across edge nodes and IoT terminals. From the evaluation results, the scalability, efficiency, and performance characteristics of CoEDMTVideo are demonstrated, offering valuable insights into its potential to revolutionize video processing tasks in collaborative edge computing settings.

The proposed CoEDMTVideo framework offers significant advantages over existing works in terms of real-time processing. Unlike the DL [16] approach, which leverages stochastic game theory and GAIL for decentralized computation offloading but struggles with the complexity of imitation learning and partial observations, CoEDMTVideo integrates collaborative edge computing and DRL. This integration allows for more efficient and timely decision-making. By leveraging the OEDQNet algorithm within the VSS, CoEDMTVideo ensures rapid processing and analysis of video data directly at the edge, reducing latency and improving the responsiveness of video streaming applications. Additionally, CoEDMTVideo's multi-driven collaborative edge computing platform enhances the computing capacity of edge nodes, enabling real-time processing even under dynamic network conditions. This advantage is particularly evident when compared to DNN-MVideo [17], which faces challenges related to the constrained computational capacities of edge devices. CoEDMTVideo overcomes these limitations by distributing computational tasks across multiple edge nodes, thereby optimizing resource utilization, and ensuring low-latency video processing. The DRL-MDP [18] approach focuses on offloading computational tasks to nearby edge servers to conserve battery life, yet it faces challenges in optimizing resource allocation and scheduling algorithms. In contrast, CoEDMTVideo's collaborative edge computing framework not only reduces energy consumption by distributing workloads across edge nodes but also enhances resource allocation using DRL algorithms. This approach minimizes the energy expenditure of individual devices while ensuring efficient processing of video tasks. Furthermore, CoEDMTVideo addresses the energy consumption challenges highlighted in DRLCO [23], particularly for UAV-enabled systems, by implementing energy-efficient DRL techniques that optimize computation offloading decisions. The framework's ability to manage and balance workloads across edge nodes results in reduced energy consumption and prolonged device battery life, making it an ideal solution for energy-constrained IoT environments. In comparison to the DLIM-CNN [19] and MEC-DIMA [21] methods, CoEDMTVideo introduces several innovative points that enhance both real-time processing and energy efficiency. DLIM-CNN employs DRL for video quality selection and radio bearer control but struggles with the complexity of training large-scale DRL models. MEC-DIMA leverages DRL for microservice caching but faces computational resource constraints in edge environments. CoEDMTVideo, on the other hand, utilizes an ensemble learning approach combined with deep Q-learning to optimize distributed model training and task allocation. This results in more efficient handling of video processing tasks and better adaptability to fluctuating network conditions. Additionally, the collaborative edge computing platform of CoEDMTVideo ensures equitable distribution of tasks, reducing the computational burden on individual edge nodes and enhancing the overall system's scalability and fault tolerance. These innovations position CoEDMTVideo as a more robust and efficient framework for managing video streaming data in dynamic IoT settings, offering significant improvements over existing methods in both real-time processing and energy efficiency.

Future work on the CoEDMTVideo framework could explore several promising extensions to further enhance its performance and adaptability. One potential direction is integrating edge intelligence techniques, such as federated learning or edge analytics, to improve robustness and scalability in dynamic IoT environments. Federated learning would allow edge devices to collaboratively train models without sharing raw data, enhancing privacy, and reducing communication overhead. Implementing edge analytics could facilitate real-time data processing and decision-making at the edge, reducing latency and improving response times. Another extension could involve developing more sophisticated resource allocation algorithms that dynamically adapt to changing network conditions and edge device capabilities, ensuring optimal performance even in highly variable environments. Investigating the impact of edge device mobility and dynamic network topologies on the CoEDMTVideo framework's performance could provide valuable insights for enhancing its resilience and efficiency in real-world deployments. Furthermore, exploring the use of advanced reinforcement learning techniques, such as multi-agent reinforcement learning, could enable more effective collaboration and coordination among edge nodes, leading to improved overall system performance. These extensions aim to address the evolving challenges of IoT video streaming and ensure the CoEDMTVideo framework remains at the forefront of edge computing innovations.

6 Conclusion

In this paper, we presented a CoEDMTVideo framework represents a significant advancement in addressing the challenges of IoT video streaming environments. By combining collaborative edge computing and DRL through the OEDQNet within the VSS, CoEDMTVideo offers a comprehensive solution to optimize distributed model training processes. This innovative approach not only streamlines decision-making and enhances system efficiency but also improves performance and scalability by effectively managing computational load and task distribution across edge nodes and IoT terminals. Through the integration of ensemble learning techniques and deep Q-learning, CoEDMTVideo sets a new standard for real-time video processing tasks in edge computing environments, promising enhanced capabilities for decision-making and resource optimization in dynamic IoT settings. Overall, the CoEDMTVideo framework demonstrates the potential to revolutionize the way video processing tasks are handled in IoT applications, mitigating challenges, such as high latency, energy consumption, and workload complexity. By leveraging cutting-edge technologies and collaborative strategies, CoEDMTVideo paves the way for more efficient and effective management of video streaming data, offering a pathway toward improved performance and scalability in real-time applications. By analyzing factors, such as average delay, bandwidth limitations, workload balance, and data communication costs, the simulations demonstrate the advantages of CoEDMTVideo over existing methods in terms of performance, efficiency, and scalability. Simulation outcomes highlight the framework's ability that obtains minimized latency, reduced energy consumption, and efficient streamline video processing tasks, ultimately leading to significantly improve decision-making processes in dynamic IoT settings. For future work, an interesting direction could be to explore the integration of edge intelligence techniques, such as federated learning or edge analytics, within the CoEDMTVideo framework. By incorporating these advanced methodologies, the impact of dynamic network conditions and edge device mobility on the performance of the framework could provide valuable insights for enhancing its adaptability and robustness in evolving edge computing scenarios.

Author contributions All the authors have participated in writing the manuscript and have revised the final version. All authors read and approved the final manuscript.

Funding There is no funding for this study.

Data Availability Not applicable.

Declarations

Conflict of interest There is no conflict of interest for this study.

Ethical Approval This article does not contain any studies with human participants and/or animals performed by any of the authors.

Consent to Participate There is no consent to participate for this study.

Consent to Publish There is no consent to publish for this study.

References

1. Laghari, A.A., Wu, K., Laghari, R.A., Ali, M., Khan, A.A.: A review and state of art of internet of things (IoT). Arch. Comput. Methods Eng. 1–9 (2021)
2. Xia, X., Chen, F., He, Q., Grundy, J., Abdelrazek, M., Jin, H.: Online collaborative data caching in edge computing. IEEE Trans. Parallel Distrib. Syst. **32**(2), 281–294 (2020)
3. Zhao, Z., Lin, P., Shen, L., Zhang, M., Huang, G.Q.: IoT edge computing-enabled collaborative tracking system for manufacturing resources in industrial park. Adv. Eng. Inform. **43**, 101044 (2020)
4. Sahni, Y., Cao, J., Yang, L., Ji, Y.: Multi-hop multi-task partial computation offloading in collaborative edge computing. IEEE Trans. Parallel Distrib. Syst. **32**(5), 1133–1145 (2020)
5. Yousefpour, A., Fung, C., Nguyen, T., Kadiyala, K., Jalali, F., Niakanlahiji, A., Kong, J., Jue, J.P.: All one needs to know about fog computing and related edge computing paradigms: a complete survey. J. Syst. Archit. **98**, 289–330 (2019)

6. Kim, Y., Song, C., Han, H., Jung, H., Kang, S.: Collaborative task scheduling for IoT-assisted edge computing. IEEE Access **8**, 216593–21606 (2020)
7. Hou, W., Wen, H., Zhang, N., Wu, J., Lei, W., Zhao, R.: Incentive-driven task allocation for collaborative edge computing in industrial internet of things. IEEE Internet Things J. **9**(1), 706–718 (2021)
8. Wang, J., Wu, W., Liao, Z., Sangaiah, A.K., Sherratt, R.S.: An energy-efficient off-loading scheme for low latency in collaborative edge computing. IEEE Access **7**, 149182–14990 (2019)
9. Dong, P., Ge, J., Wang, X., Guo, S.: Collaborative edge computing for social internet of things: applications, solutions, and challenges. IEEE Trans. Comput. Soc. Syst. **9**(1), 291–301 (2021)
10. Chen, J., Chen, S., Wang, Q., Cao, B., Feng, G., Hu, J.: iRAF: a deep reinforcement learning approach for collaborative mobile edge computing IoT networks. IEEE Internet Things J. **6**(4), 7011–7024 (2019)
11. Ren, J., Wang, H., Hou, T., Zheng, S., Tang, C.: Collaborative edge computing and caching with deep reinforcement learning decision agents. IEEE Access **8**, 120604–120612 (2020)
12. Chen, J., Ran, X.: DL with edge computing: a review. Proc. IEEE **107**(8), 1655–1674 (2019)
13. Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proc. IEEE **107**(8), 1738–1762 (2019)
14. Merluzzi, M., Di Lorenzo, P., Barbarossa, S., Frascolla, V.: Dynamic computation offloading in multi-access edge computing via ultra-reliable and low-latency communications. IEEE Trans. Signal Inf. Process. Netw. **6**, 342–356 (2020)
15. Avasalcai, C., Tsigkanos, C., Dustdar, S.: Decentralized resource auctioning for latency-sensitive edge computing. In: 2019 IEEE International Conference on Edge Computing (EDGE), pp. 72–76. IEEE (2019)
16. Wang, X., Ning, Z., Guo, S.: Multi-agent imitation learning for pervasive edge computing: a decentralized computation offloading algorithm. IEEE Trans. Parallel Distrib. Syst. **32**(2), 411–425 (2020)
17. Li, H., Ota, K., Dong, M.: Learning IoT in edge: DL for the Internet of Things with edge computing. IEEE Netw. **32**(1), 96–101 (2018)
18. Sun, H., Yu, Y., Sha, K., Lou, B.: mVideo: edge computing based mobile video processing systems. IEEE Access **8**, 11615–11623 (2019)
19. Wu, W., Gao, Y., Zhou, T., Jia, Y., Zhang, H., Wei, T., Sun, Y.: Deep reinforcement learning -based video quality selection and radio bearer control for mobile edge computing supported short video applications. IEEE Access **7**, 181740–18179 (2019)
20. Naveen, S., Kounte, M.R., Ahmed, M.R.: Low latency DL inference model for distributed intelligent IoT edge clusters. IEEE Access **9**, 160607–160621 (2021)
21. Tian, H., Xu, X., Lin, T., Cheng, Y., Qian, C., Ren, L., Bilal, M.: DIMA: distributed cooperative microservice caching for internet of things in edge computing by deep reinforcement learning. World Wide Web **25**(5), 1769–1792 (2022)
22. Han, S.Y., Lee, H.W.: Deep reinforcement learning based edge computing for video processing. ICT Express **9**(3), 433–438 (2023)
23. Huda, S.A., Moh, S.: Deep reinforcement learning -based computation offloading in UAV swarm-enabled edge computing for surveillance applications. IEEE Access **1**, 68269–68285 (2023)
24. Qian, B., Yubo, X., Di, W., Zhenyu, W., Renyu, Y., Shibo, H., Jiming, C., Rajiv, R.: Edge-cloud collaborative streaming video analytics with multi-agent deep reinforcement learning. IEEE Netw. **39**(1), 165–173 (2024)
25. Pham-Quoc, C.: HLS and FPGA-powered streaming video encoder accelerator for IoTs edge computing. J. Adv. Inf. Technol. **15**(1), 59–65 (2024)
26. Yang, J., JiaLin, L., Xu, Z., ShaoBo, L., ChuanYue, X., Jianjun, H.: HA-A2C: hard attention and advantage actor-critic for addressing latency optimization in edge computing. IEEE Trans. Green Commun. Netw. **9**, 207–217 (2024)
27. Petersohn, D., Macke, S., Xin, D., Ma, W., Lee, D., Mo, X., Gonzalez, J.E., Hellerstein, J.M., Joseph, A.D., Parameswaran, A.: Towards scalable dataframe systems (2020). arXiv:2001.00888
28. Reyad, M., Sarhan, A.M., Arafa, M.: A modified Adam algorithm for deep neural network optimization. Neural Comput. Appl. **25**, 1–8 (2023)
29. Mai, T., Jaillet, P.: A relation analysis of Markov decision process frameworks (2020). arXiv:2008.07820
30. Ororbia, M.E., Warn, G.P.: Design synthesis through a Markov decision process and Reinforcement Learning framework. J. Comput. Inf. Sci. Eng. **22**(2), 021002 (2022)
31. Loh, F., Wamser, F., Poignée, F., Geißler, S., Hoßfeld, T.: Youtube dataset on mobile streaming for internet traffic modelling and streaming analysis. Sci. Data **9**(1), 293 (2022)
32. Ranjani, J., Sheela, A., Meena, K.P.: Combination of NumPy, SciPy and Matplotlib/Pylab-a good alternative methodology to MATLAB-A comparative analysis. In: 2019 1st International Conference on Innovations in Information and Communication Technology (ICIICT), pp. 1–5. IEEE (2019)
33. Li, S.E.: Deep reinforcement learning. In: Reinforcement Learning for Sequential Decision and Optimal Control, pp. 365–402. Springer Nature Singapore, Singapore (2023)
34. Nauman, A., Qadri, Y.A., Amjad, M., Zikria, Y.B., Afzal, M.K., Kim, S.W.: Multimedia internet of things: a comprehensive survey. IEEE Access **8**, 8202–8250 (2020)
35. Ji, W., Xu, J., Qiao, H., Zhou, M., Liang, B.: Visual IoT: enabling internet of things visualization in smart cities. IEEE Netw. **33**(2), 102–110 (2019)
36. Carvalho, G., Cabral, B., Pereira, V., Bernardino, J.: Edge computing: current trends, research challenges and future directions. Computing **103**, 993–1023 (2021)
37. Sufyan, F., Banerjee, A.: Computation offloading for distributed mobile edge computing network: a multi objective approach. IEEE Access **8**, 149915–149930 (2020)
38. Li, M., Gao, J., Zhao, L., Shen, X.: Deep reinforcement learning for collaborative edge computing in vehicular networks. IEEE Trans. Cogn. Commun. Netw. **6**(4), 1122–1135 (2020)
39. Jiang, X., Yu, F.R., Song, T., Leung, V.C.: A survey on multi-access edge computing applied to video streaming: some research issues and challenges. IEEE Commun. Surv. Tutor. **23**(2), 871–903 (2021)
40. Chen, Y., Sun, Y., Feng, T., Li, S.: A collaborative service deployment and application assignment method for regional edge computing enabled IoT. IEEE Access **8**, 112659–112673 (2020)
41. Chen, Y., Liu, Z., Zhang, Y., Wu, Y., Chen, X., Zhao, L.: Deep reinforcement learning-based dynamic resource management for mobile edge computing in industrial internet of things. IEEE Trans. Ind. Inf. **17**(7), 4925–4934 (2020)

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Shaharyar Alam Ansari A distinguished academician and computer science expert, currently serves as an Assistant Professor at Bennett University, specializing in Computer Science Engineering and Technology, located in Greater Noida, India. In addition to his role at Bennett University, he contributes his expertise as a visiting faculty member in Computer Science at the Residential Coaching Academy, Aligarh Muslim University, Aligarh. Before joining Bennett University, he served as an Assistant Professor at Sharda University in the Department of Computer Science and Application, leaving an indelible mark on the academic community. Dr. Ansari's academic journey has been marked by a commitment to excellence. He earned his Ph.D. in Computer Science from Aligarh Muslim University (AMU), solidifying his dedication to advancing knowledge in his field. Acknowledging his academic prowess, Dr. Ansari was honored with the prestigious Maulana Azad National Fellowship from the University Grants Commission (UGC), Government of India. His scholarly achievements extend to qualifying the esteemed NTA UGC-NET national examination in 2020 and 2021, attesting to his comprehensive understanding of computer science. Notably, he has authored a book tailored for MCA and M.Sc. (Data Science & Cybersecurity) aspirants, further contributing to the academic literature available on platforms, such as Flipkart and Amazon. Dr. Ansari's research interests encompass cutting-edge fields, such as Image Processing, Video Analytics, Soft Computing, and Deep Learning. In the realm of academia, his areas of teaching expertise span Data Structures, Operating Systems, Computer Networks, Database Management Systems, and Computer Programming. Beyond his academic pursuits, Dr. Ansari's multidimensional contributions to education, research, and literature make him a notable figure in the vibrant landscape of computer science academia in India.



Mohammad Luqman He has received his Master's in Computer Science and Applications from Aligarh Muslim University, Aligarh, India in 2017. Currently, he is pursuing Ph.D. in Information security from the same institution. He is a JRF awardee and a MANF, national award, recipient. His current research interests include cryptography, information security, deep learning, soft computing, wireless networks etc.



Dr. Mohd Anas Wajid He is an Assistant Professor at Galgotias University, Greater Noida, India, in the Department of Computer Science & Application. He received his Ph.D. degree in Computer Science from Aligarh Muslim University, India. He was awarded with the MITACS-SICI Globalink Research Award by Mitacs in collaboration with HRD ministry, Government of India to do a project at the University of Athabasca, Edmonton, Alberta, Canada. ACM India Council named him an ACM India Anveshan Setu Fellow, and he received a fellowship to conduct a part of his research at IIIT-Delhi. He was also a recipient of Maulana Azad National Fellowship (SRF) from UGC, Government of India. He qualified various prestigious national exams such as UGC-NET and GATE multiple times. His contribution to Neutrosophic research earned him a Diploma from the Neutrosophic Science International Association (NSIA), University of New Mexico, United States (USA). He has several research papers to his credit in refereed journals and conferences of international repute, such as Journal of Cloud Computing, Nature Scientific Reports, Journal of Computational Intelligence & Neuroscience, and Neutrosophic Sets & Systems. He has co-authored one book and two patents. He has keen interest in Soft Computing, Machine Learning, Data Science, Information Retrieval, and Neutrosophy. He has academic as well as industrial experience.



Mohammad Saif Wajid He has received Bachelor's degree in Computer Science & Engineering from Dr. A.P.J. Abdul Kalam Technical University, India, and Masters's Degree from BBD University, India. He is currently enrolled as a Full-Time Ph.D. student at Tecnologico de Monterrey, Monterrey Campus, Mexico. He works as a Visiting Research Scholar at the University of Texas, San Antonio, USA. For the past 08 years, he has been working as an Assistant Professor in Computer Science & Engineering and worked as a Software Engineer for 01 years in India. His research jumps around the potential applications of Digital Twin, Knowledge Graph, Machine Learning, Computer Vision, NLP, and Neutrosophy, touching areas, such as Violence Detection, Healthcare Systems, Dense Captioning of images/videos, and DTWIN for Smart cities. He is passionate about digital learning, real-time and asynchronous, and an early adopter of emerging software and hardware solutions to facilitate teaching and research. His other favored tool is LaTeX, which he likes to use for all academic writings and presentations. He has published over 25 research papers in international journals, conferences, and 01 Book. He has supervised 08 Master's Thesis.



Shahnawaz Ahmad He has received the B.Tech degree in Computer Science and Engineering from GBTU, India, in 2012, the M.Tech degree in Computer Engineering from MDU Rohtak, India, in 2015, and a Ph.D. in Engineering and Technology from Jamia Millia Islamia in 2023. He is currently an Assistant Professor at Bennett University, Greater Noida, India. With over 50 research publications in international and national conferences and esteemed journals, he has established himself in academia. He has served as the Track Chair for two flagship international conferences held in 2023, both of which were technically co-sponsored by IEEE. His expertise focuses on Cloud Computing, Artificial Intelligence, and Machine Learning, particularly on CASB and key management platforms for cloud security.